Министерство образования и науки Российской Федерации

Санкт-Петербургский политехнический университет
Петра Великого



И. В. Никифоров, Н. В. Воинов, П. Д. Дробинцев

НАУКА О ДАННЫХ И АНАЛИТИКА БОЛЬШИХ ОБЪЕМОВ ИНФОРМАЦИИ

КУРСОВОЕ ПРОЕКТИРОВАНИЕ

Учебное пособие

ПОЛИТЕХ-ПРЕСС
Санкт-Петербургский
политехнический университет
Петра Великого

Санкт-Петербург
2021

УДК 378.241 (075.8)
ББК 32.973я73
Н62

Р е ц е н з е н т ы:
Доктор технических наук, профессор, профессор ВШ ПИ
Санкт-Петербургского политехнического университета Петра Великого
С. М. Устинов
Доктор технических наук, профессор, профессор ВШ ПИ
Санкт-Петербургского политехнического университета Петра Великого
С. А. Молодяков

Никифоров И. В. **Наука о данных и аналитика больших объемов информации. Курсовое проектирование** : учеб. пособие / И. В. Никифоров, Н. В. Воинов, П. Д. Дробинцев. – СПб. : ПОЛИТЕХ-ПРЕСС, 2021. – 93 с.

Приведены материалы, раскрывающие содержание, методику подготовки и оформления курсового проекта студентами, изучающими учебную дисциплину «Наука о данных и аналитика больших объемов информации», входящую в состав основной образовательной программы направления подготовки магистров 09.04.04 «Программная инженерия». Представлены упражнения по основным инструментальным средствам, примеры тестовых вопросов для контроля теоретических знаний и сведения по организации руководства студентами и порядку защиты курсового проекта.

Учебное пособие предназначено для студентов, обучающихся по направлению подготовки магистров 09.04.04 «Программная инженерия».

Печатается по решению
Совета по издательской деятельности Учёного совета
Санкт-Петербургского политехнического университета Петра Великого

ISBN 978-5-7422-7352-3

© Никифоров И. В., Воинов Н. В.,
Дробинцев П. Д., 2021
© Санкт-Петербургский политехнический
университет Петра Великого, 2021

ОГЛАВЛЕНИЕ

Список сокращений ……..... 4
Введение …….. 5
1. Выбор предметной области …….……. 9
2. Формирование набора данных …….…… 14
3. Архитектура проектируемой системы…….. 16
4. Хранение данных……..…….. 19
5. Обработка данных……..……. 23
5.1. Подход решения прогностической задачи.....……... 29
5.2. Пример решения задачи поиска некорректных и ошибочных данных ……..……..……..... 30
6. Визуализация результатов……..……..…….. 36
7. Программная реализация …….. 38
8. Защита проекта ……... 38
9. Правила формирования итоговой оценки ……... 41
Учебная литература …… 42
Приложения ……. 43
Приложение 1. Образец оформления титульного листа……. 43
Приложение 2. Упражнения по технологии Apache Hadoop….. 45
Упражнение 1 – установка Hadoop и работа с HDFS……... 47
Упражнение 2 – разработка Map-Reduce задачи…….. 50
Упражнение 3 – создание базы данных в HBase…….. 54
Приложение 3. Упражнения по технологии Apache Spark…….. 58
Упражнение 1 – установка Spark и работа с Spark Shell….. 59
Упражнение 2 – создание и операции над RDD……... 62
Упражнение 3 – создание и работа с DataFrame……... 65
Приложение 4. Примеры итогового тестового задания ……. 68
Итоговый тест – Вариант 1…..…..…..…..…..…..…..…... 68
Итоговый тест – Вариант 2…..…..…..…..…..…..…..…... 79
Приложение 5. Примеры экзаменационных вопросов……. 90

СПИСОК СОКРАЩЕНИЙ

БД — база данных;
ИТ — информационные технологии;
ОС — операционная систем;
СХД — система хранения данных;
AMQP — продвинутый протокол передачи сообщений (Advanced Message Queuing Protocol);
API — программный интерфейс приложения (Application Program Interface);
AWS — Amazon Веб-Сервисы (Amazon Web Services);
CSV — разделённые запятыми значения (Comma-Separated Values);
DFS — распределенные файловые системы (Distributed File System) ;
ETL — извлечь, трансформировать, загрузить (Extract, Transform, Load);
HDFS — распределенная файловая система Hadoop (Hadoop Distributed File System);
JSON — объектная нотация JavaScript (JavaScript Object Notation);
HTTP — протокол передачи гипертекста (Hypertext Transfer Protocol);
HTML — язык гипертекстовой разметки (Hypertext Markup Language);
RDD — устойчивый распределенный набор данных (Resilient Distributed Dataset);
SQL — язык структурированных запросов (Structured Query Language);
XML — расширяемый язык разметки (Extensible Markup Language);
URL — единообразный локатор ресурса (Uniform Resource Locator).

ВВЕДЕНИЕ

Учебная программа дисциплины «Наука о данных и аналитика больших объемов информации», входящей в учебный план подготовки магистров по направлению 09.04.04 «Программная инженерия», предусматривает выполнение студентами курсового проекта, работа над которым проводится в течение 1-го семестра.

Целью курсового проектирования является формирование навыков решения прикладной задачи анализа информации для выбранной предметной области с применением современных подходов, методов и инструментов работы с данными.

Изучение дисциплины «Наука о данных и аналитика больших объемов информации» предполагает, что студенты уже имеют базовые навыки проектирования и разработки программных систем и знакомы на уровне понятий, терминов и технологий разработки с перечисленными ниже областями знаний:

- «Объектно-ориентированное программирование»;
- «Базы данных»;
- «Математическое моделирование»;
- «Теория вероятности»;
- «Статистическое моделирование»;
- «Технология параллельного программирования».

Выполнение курсового проекта базируется на сформированных у студентов при обучении в бакалавриате по направлению подготовки 09.03.04 «Программная инженерия» способностей формализовать предметную область программного проекта и разработать спецификации для компонентов программного продукта, способностью готовить презентации, оформлять научно-технические отчеты по результатам выполненной работы, готовностью к использованию методов и инструментальных средств исследования объектов профессиональной деятельности и умениях применять подходы моделирования, анализа и использования формальных методов конструирования программного обеспечения.

В частности, для успешного выполнения курсового проекта ожидается, что студенты уже умеют применять принципы проектирования

SOLID[1], ACID[2] и паттернов программирования на практике, в том числе и при создании сложных Enterprise-приложений. Также студенты должны владеть знаниями и обладать навыками тестирования программного обеспечения на уровне написания модульных и интеграционных тестов.

Единой задачей курсового проектирования является приобретение студентами компетенций владения методами и средствами получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе в глобальных компьютерных сетях, а также владением существующими методами и алгоритмами решения задач распознавания и обработки данных, включающих в себя как основную составляющую приобретение опыта проектировать аналитические системы с параллельной обработкой.

Курсовое проектирование по дисциплине «Наука о данных и аналитика больших объемов информации» решает также такие задачи, как формирование умений студентов работать с технической литературой, самостоятельно разрабатывать программные средства и принимать решения, отвечать на поставленные в задании вопросы, делать выводы и демонстрировать полученный результат.

Курсовой проект состоит из следующих основных частей:

— выбор предметной области;

— формулирование задач анализа и выбор используемых технологий;

— сбор и получение информации;

— хранение данных;

— обработка данных;

— визуализация результатов анализа;

— подготовка презентации и пояснительной записки.

[1] S.O.L.I.D.— пять принципов проектирования классов в ООП. S - Single responsibility principle,O - Open/closed principle, L - Liskov substitution principle, I - Interface segregation principle, D - Dependency inversion principle

[2] A.C.I.D. — это 4 принципа проектирования транзакционных систем, аналог S.O.L.I.D. для баз данных. Atomity (A), Consistency (C), Isolation (I), Durability (D).

Выполнение курсового проекта осуществляется индивидуально каждым студентом. Групповая подготовка курсового проекта по одной теме не допускается.

Курсовой проект должен быть выполнен в сроки, установленные кафедрой. Отчет по курсовому проекту должен удовлетворять требованиям кафедры по оформлению курсовых работ и проектов.

Тематика курсового проекта может соответствовать направленности выпускной квалификационной работе студента, выполненной им при обучении в бакалавриате или являться началом магистерского исследования студента, поэтому выбор темы курсового проекта имеет важное значение в деле решения задач, поставленных учебной программой рассматриваемой дисциплины.

Структурно учебное пособие «Курсовое проектирование» построено в соответствии с последовательностью выполнения студентами проекта и включает 12 разделов и 4 приложения, перечень которых приведен в оглавлении.

В разделе «Введение» приведено описание цели курсового проекта и сведений о необходимой квалификации студентов, приступающих к выполнению проекта.

Раздел «Выбор предметной области», «Формирование набора данных», «Архитектура проектируемой системы», «Хранение данных», «Обработка данных», «Визуализация результатов» и «Программная реализация» содержит указания к выполнению каждого из этапов и компонента проекта.

Требования к презентации результатов проекта и написанию итогового отчета приведены в разделе «Защита проекта».

В разделе «Правила формирования итоговой оценки» раскрыты критерии оценки сложности и качества проделанной студентом работы.

Отдельным разделом в учебном пособии выделена «Учебная литература», где представлены учебные и научные издания, которые студент должен использовать при разработке темы и создании программной реализации проекта. Указания на адреса сведений о программах, инструментах и фреймворках приведены в постраничных ссылках.

В приложении 1 «Образец оформления титульного листа» представлен шаблон титульного листа отчета по курсовому проекту, который студент должен использовать.

Реализация задач получения практических навыков использования инструментов и освоения фреймворков Apache Hadoop, Apache Spark необходима для выполнения курсового проекта и требует навыков администрирования сложных программных систем, программирования и работ с базами данных. С этой целью в пособии приводится ряд упражнений, приведенных в Приложениях 2, 3, выполнив которые студент достигает поставленных задач. Выполнение упражнений рекомендуется выполнять после раздела «Формирование набора данных». При этом трудоемкость выполнения упражнений не включена в общую трудоемкость подготовки и защиты курсового проекта, утвержденного учебным планом. В описании упражнений приводятся краткие теоретические сведения.

Завершающим разделом пособия является приложение 4, в котором приводится пример итогового теста для проверки теоретических знаний студентов по учебной дисциплине.

1. ВЫБОР ПРЕДМЕТНОЙ ОБЛАСТИ

Курсовой проект не привязан к какой-либо конкретной предметной области. Предполагается отойти от принципа выполнения заранее поставленных и четко сформулированных задач, чтобы предоставить исполнителю гибкость и возможность творческого подхода выполнения. Таким образом, исполнителю предоставляется возможность самостоятельного выбора интересующей его прикладной области, над которой в течение семестра будет проводиться работа. Если же исполнитель не имеет своих собственных предпочтений, то ему предлагается на выбор предметные области, перечисленные ниже:

— «Анализ данных социальных сетей». Например, электронные ресурсы Vkontakte[3], Twitter[4], Facebook[5], LinkedIn[6] и др.;

— «Анализ рынка вакансий». Например, электронный ресурс HeadHunter[7];

— «Анализ фильмов». Например: интернет-проект «Кинопоиск»[8];

— «Анализ журнала запросов к сайту Wikipedia[9]»;

— «Технический радар». Анализ информации с ресурса StackOverFlow[10];

---

[3] Vkontakte. [Электронный ресурс]. Режим доступа: // http://www.vk.com.

[4] Twitter. [Электронный ресурс]. Режим доступа: // http://www.twitter.com.

[5] Facebook. [Электронный ресурс]. Режим доступа: // http://www.facebook.com.

[6] LinkedIn. [Электронный ресурс]. Режим доступа: // http://www.linkedin.com.

[7] HeadHunter — *качественная база резюме* и *вакансий* и *современные сервисы* для поиска работы и персонала. [Электронный ресурс]. Режим доступа: // http://www.hh.ru.

[8] Кинопоиск — русскоязычный интернет-проект, посвящённый кинематографу, [Электронный ресурс]. Режим доступа: // http://www.kinopoisk.ru.

[9] Wikipedia — свободная общедоступная мультиязычная универсальная интернет-энциклопедия, [Электронный ресурс]. Режим доступа: // http://www.wikipedia.org.

[10] StackOverFlow — популярная система вопросов и ответов о программировании, [Электронный ресурс]. Режим доступа: // http://www.stackoverflow.com.

— «Использование существующих решений и наборов данных». Например, информация с ресурса Kaggle[11] (см. условия выставления итоговой оценки). Например, «задача Титаника»[12].

Приветствуются темы курсовых работ из следующих областей: «Образование», «Наука», «Здравоохранение», «Информационные технологии» (ИТ), и др.

Утверждение направления проведения работы, ее предметной области и содержательное направление осуществляет руководитель курсового проекта с каждым исполнителем индивидуально.

Для выбранной предметной области требуется сформулировать от 5 до 20 задач для проведения анализа. Задачи могут быть отнесены к следующим областям анализа: анализ социальных сетей (Social Mining), анализ Интернет-ресурсов (Web Mining), анализ текста (Text Mining), анализ данных (Data Mining). Классификация задач анализа по областям приведена на рис.1.

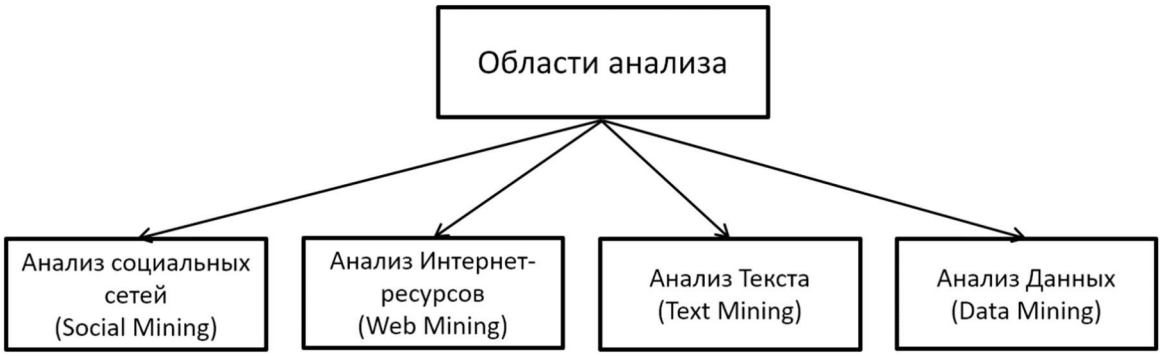


Рис.1 Классификация задач анализа по областям

В тоже время задачи анализа можно классифицировать по типу: задачи статистического типа и задачи исследовательского типа. Классификация приведена на рис.2.

[11] Kaggle - англоязычный ресурс, посвященный задачам анализа и науке о данных, [Электронный ресурс]. Режим доступа: // http://www.kagle.com.

[12] «Тита́ник» (англ. *Titanic*) — британский трансатлантический пароход. «Задача Титаника» - создание модели для предсказания выживших пассажиров парохода в зависимости от характеристик пассажира: его пол, возраст, номер каюты и т. д..

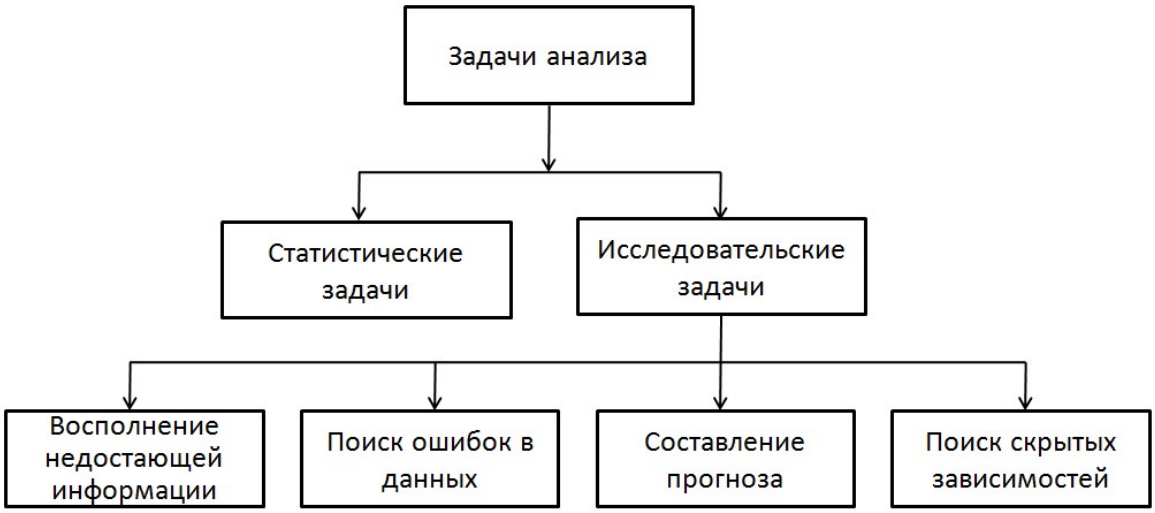


Рис. 2. Классификация задач анализа по типу

Статистические задачи относятся к традиционной обработке известного набора данных, объектов и их атрибутов для получения численных характеристик. Традиционно принято считать, что статистические задачи относятся к категории бизнес-аналитики (Business Intelligence). Они призваны помочь ответить на вопросы: «Какие численные показатели получила отрасль за прошлое время?», «Как правильно настроить рабочие процессы на основе прошлых, исторических данных?». Иными словами, результаты решения статистических задач помогают понять, что же произошло в прошлом и как на основе этих данных оптимизировать бизнес или производственные процессы и получить выгоду, зачастую экономическую. Особенностью реализации этого типа задачи являются: большое количество записей, большой объем информации и реализация алгоритмов обработки средствами и фреймворками для высокопроизводительных и распределенных вычислений.

Исследовательские задачи (Data Science), в отличие от статистических, подразумевают поиск скрытых зависимостей и паттернов в данных, восстановление недостающей информации, поиск ошибок в данных, а также составление некоторых прогнозов на будущее. Особенностью этого типа задач является использование инновационных, современных и прогрес-

сивных методов анализа, которые в том числе позволяют построить своего рода экспертную систему.

При формулировании задач анализа необходимо, чтобы были представлены на утверждение задачи из каждой категории. Проработка каждой задачи анализа требует проявления фантазии и собственной заинтересованности в получении ответа на поставленный вопрос, потому что именно личностная заинтересованность может привести к высокому качеству выполнения проекта. Самостоятельная проработка предметной области и сценариев решения позволяет развить у исполнителя умения самостоятельного формирования технического задания, т. е. тех компетенций, которыми должен обладать выпускник направления «Программная инженерия».

Стоит принимать во внимание, что данные, подвергаемые анализу, могут обладать рядом неприятных свойств: неполнота, противоречивость, некорректность и разнородность. Если не учитывать возможность наличия таких свойств в данных, то результаты решения задач анализа могут находиться в другой плоскости относительно истинного решения. Для того, чтобы результаты решения задач были корректными, необходимо осуществлять валидацию и верификацию подвергаемой анализу информации. Зачастую применяют следующие подходы для проверки данных на корректность: методы машинного обучения, поиск нечетких связей и соответствий, и выявление обратной связи между атрибутами объектов, результатами решения задачи и входных данных.

Перед началом работы по конкретно сформулированной задаче анализа необходимо убедиться, что в открытом доступе не опубликовано похожих исследований, а если они и опубликованы, то результаты сторонних исследований необходимо использовать для проверки собственного решения.

Если рассматривать предметную область «Вакансии» с web-ресурса «HeadHunter», то в роли задач анализа могут выступать далее приведенные статистические и исследовательские задачи.

Статистические задачи:

— анализ наиболее востребованных на рынке информационных технологий языков программирования в заданные интервалы времени (начиная с 2002 по 2016 гг.);

— определение распределения вакансий в области информационных технологий по регионам в зависимости от года;

— поиск наиболее популярных профессий в Российской Федерации;

— нахождение зависимости зарплаты от специализации;

Исследовательские задачи:

— поиск скрытых зависимостей между характеристиками работодателя и представленных вакансий;

— прогнозирование заработной платы в области IT на 2030 год.

Для предметной области «Социальные сети» в роли статистических задач анализа могут выступать:

— определение перечня городов, из которых в вузы Санкт-Петербурга приезжают для поступления абитуриенты, в том числе и зарубежные;

— нахождения перечня стран и городов, в которых работают выпускники вузов Санкт-Петербурга;

— установление параметров корреляции популярных тем обсуждений в социальных сетях с событиями в новостях.

Исследовательскими задачами для социальных сетей могут быть:

— прогнозирование количества приезжих абитуриентов в вузы Санкт-Петербурга;

— поиск скрытых зависимостей между родным городом абитуриента и Санкт-Петербургом.

Перечисленные выше примеры задач анализа могут показаться достаточно простыми и требующими создания одного или нескольких запросов к базам данных (БД). Исполнителю нужно сформулировать задачи анализа разной сложности, чтобы каждая из задач решалась с использованием разных подходов и методов обработки информации. Степень сложности сформулированных задач влияет на итоговую защиту проекта.

2. ФОРМИРОВАНИЕ НАБОРА ДАННЫХ

Во время выполнения курсового проекта может потребоваться работать с информацией разного типа. Традиционно принято выделять четыре типа данных: структурированные данные, полуструктурированные данные, квазиструктурированные данные и не структурированные данные. На рис. 3 приведена классификация информации по типам. При этом стоит отметить, что объем занимаемых данных растет с уменьшением их структурированности.

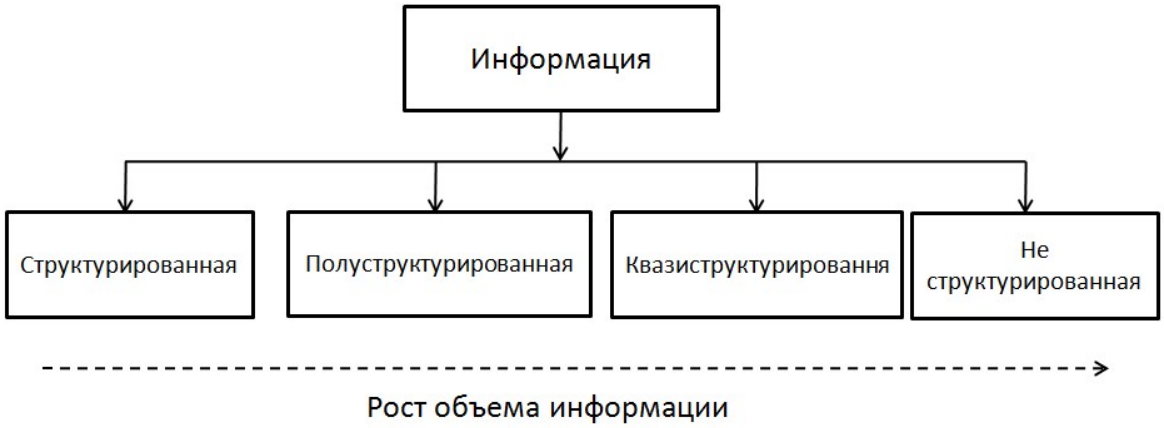


Рис. 3. Классификация информации по типу

Исполнитель самостоятельно выбирает тип данных с которым в дальнейшем будет работать, но требуется принимать во внимание, что поскольку в учебной дисциплине «Наука о данных и аналитика больших объемов информации» рассматриваются подходы и технологии обработки именно большого объема данных, то для выбранной прикладной области предполагается иметь для проведения анализа не менее 2 Гб структурированных или полуструктурированных данных, если не используются методы анализа неструктурированного контента. В случае, если используются методы анализа неструктурированного контента, такого как изображения, аудио- и видеозаписи, то минимальный объем информации устанавливается равным 5Гб.

Особо стоит отметить, что в некоторых случаях обеспечить удовлетворение критерия по объему данных невозможно в связи с природными и техническими ограничениями на количество предоставляемых данных ис-

точником. В этом случае от исполнителя требуется обоснование полученного исходного набора информации.

Важным аспектом используемых для анализа данных является полнота анализируемой выборки. Исполнитель должен предоставить доказательство того, что исходная выборка достаточна для проведения исследований и сбора статистических данных.

В курсовом проекте не предусмотрено предоставление готового набора данных (англ., Data Set) для решения сформулированных задач анализа. Студенту требуется самостоятельно сформировать собственный набор данных. Такой подход позволит сделать каждую работу индивидуальной и уникальной. При этом этап сбора данных является одним из самых трудоемких этапов курсового проекта.

Помочь в формировании набора данных могут web-ресурсы, предоставляющие программный API для получения информации в виде форматов xml, json или любом другом формате.

Одним из таких ресурсов является Интернет-ресурс Head Hunter, предоставляющий информацию по работодателям, вакансиям и соискателям. На рассматриваемом ресурсе в разделе для разработчиков представлено качественное и детальное описание того, как использовать программные интерфейсы (API) сервисов для получения данных, какие типы HTTP-запросов необходимо отправлять на какие адреса (URL). С использованием программного интерфейса ресурса Head Hunter можно сформировать набор данных до 100 Гб информации, который содержит приблизительно 17 000 000 вакансий за временной промежуток от 2002 года до 2017 года, а также информацию приблизительно о 800 000 компаниях. Этого набора данных вполне достаточно для выполнения курсового проекта.

Для поиска других ресурсов, предоставляющих программные интерфейсы получения информации, рекомендуется использовать сервисы-агрегаторы[13]. Такие сервисы обычно хранят перечень сайтов, для которых доступны программные интерфейсы. Предпочтение стоит отдавать национальным ресурсам, нежели зарубежным.

---

[13] Одним из таких сервисов-агрегаторов является ресурс, http://www.programmableweb.com.

Еще одним способом получения информации с Интернет ресурсов является технологии web-scraping[14], позволяющие из HTML-разметки страниц сайтов получать информацию в виде csv, xml или других форматов. К технологиям web-scrapping можно отнести: import.io[15], Jaunt[16], JSoup[17] и другие.

3. АРХИТЕКТУРА ПРОЕКТИРУЕМОЙ СИСТЕМЫ

Курсовой проект подразумевает создание программного решения, позволяющего автоматически или полуавтоматически решать сформулированные задачи анализа. В основе решения может быть заложена относительно простая, но функциональная и расширяемая модульная схема.

Стоит отметить, что программное решение должно обладать хорошей производительностью, гибким масштабированием, быть распределенным и гарантировать надежность передачи данных между узлами системы. Еще одной важной особенностью рассматриваемой архитектуры является возможность гибкой настройки проводимых в системе аналитик. На рис.4 представлена рекомендуемая модульная схема разрабатываемой системы.

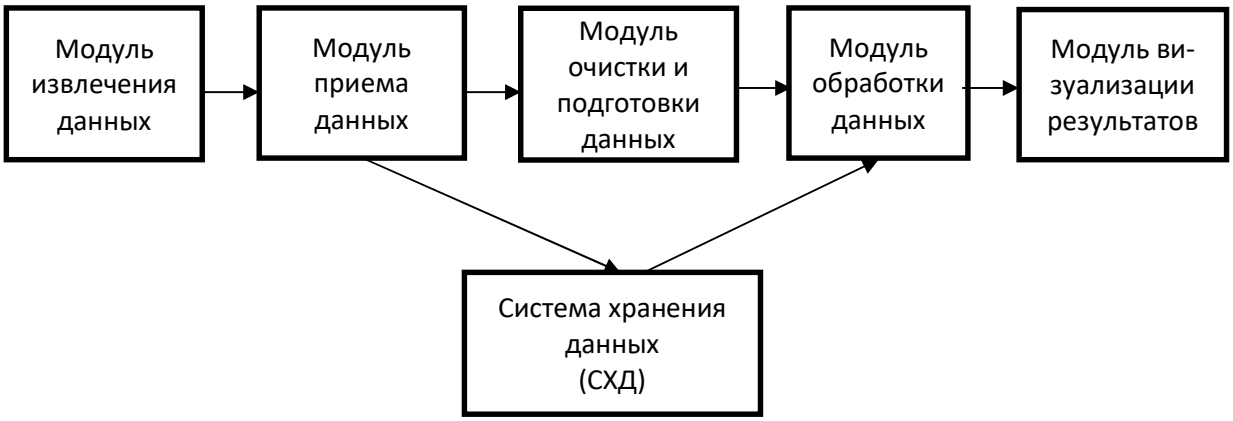


Рис. 4. Модульная схема программной системы анализа данных

Представленный шаблон архитектуры не является обязательным и строго зафиксированным и может видоизменяться в зависимости от выбранной задачи анализа и используемых технологий и методов. Таким об-

[14] Web-Scraping - компьютерная технология, позволяющая извлекать информацию с Интернет ресурсов.
[15] Import.io. [Электронный ресурс]. Режим доступа: https://www.import.io/
[16] Jaunt. [Электронный ресурс]. Режим доступа: http://jaunt-api.com/
[17] Jsoup. [Электронный ресурс]. Режим доступа: https://jsoup.org/

разом, за исполнителем курсового проекта остается право видоизменять схему, варьировать количеством модулей и связей между модулями в тех границах, которые позволяют решить поставленную задачу и удовлетворить требованиям на систему, перечисленным выше.

Использование «Модуля извлечения данных» осуществляет получение данных из локального, внешнего или удаленного источника и передачу их в систему. Например, если сформированный набор данных находится на локальном диске, то рассматриваемый модуль просто считывает информацию с локального диска и передает ее в модуль приема. Если набор данных формируется из удаленного источника, то модуль извлечения осуществляет соединение с удаленным источником и извлекает информацию из него по средствам предоставляемого API удаленного источника или любого поддерживаемого протокола обмена данными. Стоит отметить, что модуль извлечения данных обладает рядом характеристик, таких как скорость извлечения и скорость передачи. В разрабатываемой системе для проведения нагрузочного тестирования итоговой системы следует предусмотреть возможность варьирования перечисленных параметров.

Поскольку предполагается, что проектируемая система должна работать с большим объемом данных и система, в теории, должна обеспечивать высокоскоростной и надежный входной поток данных, то «модуль приема данных» должен гарантировать получение информационных пакетов от «модуля извлечения», их доставку в систему хранения данных (СХД) и отсутствие потерянных пакетов. Для удовлетворения этому требованию возможно использование брокеров сообщений в модуле приема данных. К таким брокерам сообщений относятся RabbitMQ[18], Kafka[19] и другие системы удовлетворяющие протоколу AMQP[20].

«Модуль очистки и подготовки данных» отвечает за очистку сырых данных, поступивших от модуля приема, их помещение в хранилище при

[18] RabbitMQ. [Электронный ресурс]. Режим доступа: https://www.rabbitmq.com/

[19] Apache Kafka. [Электронный ресурс]. Режим доступа: http://*kafka.apache.org/*

[20] AMQP - Advanced Message Queuing Protocol. [Электронный ресурс]. Режим доступа: http://www.amqp.org/

необходимости, или, в случае обработки в режиме реального времени их передачу непосредственно обработчику.

Зачастую три модуля в совокупности: извлечение, прием и очистка, формируют процесс, известный как ETL-процесс. Существует огромное число готовых инструментальных средств, которые поддерживают данный процесс. Например: Talend Open Studio, Data Science Studio, GeoKettle и др. При выполнении курсового проекта целесообразно использовать готовые ETL-инструменты для реализации поставленных задач. Но обязательным является условие сохранения сформулированных свойств системы по производительности, масштабируемости и надежности.

«Модуль обработки данных» в зависимости от сценария использования системы получает данные из «Хранилища» или непосредственно с модуля очистки и подготовки. Он содержит основную логику обработки данных с использованием, например, Spark[21] и методов машинного обучения.

После обработки данные передаются на «модуль визуализации результатов», который осуществляет отображение результатов обработки в виде диаграмм, графиков или может даже наносить особые отметки на интерактивную карту.

Проект по аналитике данных предполагает не линейное выполнение, а итерационную работу, то есть на каждом этапе могу быть выявлены недочеты, исправление которых требует возвращения к предыдущему этапу. Например, на этапе реализации алгоритма анализа в модуле обработки может стать очевидным, что используемая модель данных не достаточно полна (а именно, требуется добавление одного или нескольких новых полей). В связи с этим требуется внести изменения в «модуль очистки и подготовки данных» или даже в «модуль извлечения данных». Исходя из этого, особо важным является продумывание архитектуры модулей с заделом на будущее расширение, сопровождение и поддержку. Еще одной причиной обратить внимание на возможность расширения системы является необходимость проведения нескольких итераций и разработки различных

[21] Spark. [Электронный ресурс]. Режим доступа: // http://spark.apache.org/

алгоритмов вычислений для поиска скрытых зависимостей и прогнозирования.

Совокупное использование представленных модулей позволяет изучить все этапы жизненного цикла проекта по аналитике, начиная с этапа сбора информации, заканчивая этапом представления результатов и внедрения готового продукта.

4. ХРАНЕНИЕ ДАННЫХ

Хранение сформированного набора данных или набора данных, который прошел предварительную очистку и готов поступить на обработку, предполагается осуществлять в базе данных.

Существует теорема CAP, являющаяся аббревиатурой от сокращения трех свойств: "Consistency" (C) - согласованность, "Availability" (A) - доступность и "Partition Tolerance" (P) - устойчивость к расщеплению. Сама теорема гласит, что ни одна система хранения данными не может удовлетворять одновременно всем трем свойствам. Из теоремы вытекает то, что все базы данных могут быть разделены на 3 типа. Первый тип, это тип, удовлетворяющий свойствам "C" и "A" (CA), второй тип - "A" и "P" (AP), третий тип - "C" и "P" (CP). На рис. 5 представлено разбиение типов хранилищ на классы в соответствии с теоремой CAP.

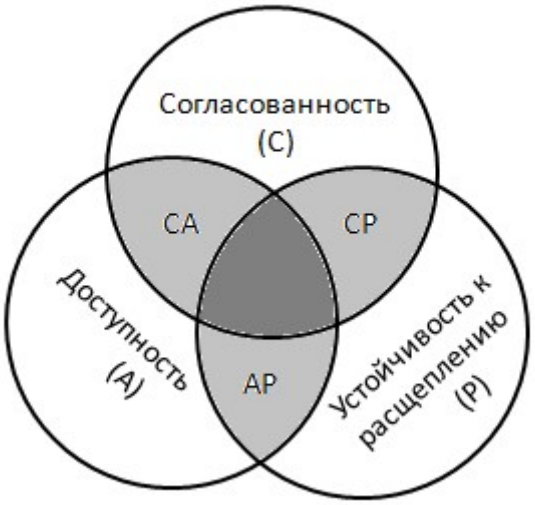


Рис. 5. Разделение хранилищ данных на классы в соответствии с теоремой CAP.

Для традиционных реляционных баз данных существует крепкая теоретическая основа. В теореме CAP. они удовлетворяют свойствам CA, из которых вытекают принципы ACID. Они гибки, надежны и относительно легко администрируются. При их использовании в производстве известны риски и ограничения. Но в тоже время, они обладают посредственной масштабируемостью на сверхвысокую нагрузку в виде большого количества запросов на чтение и запись. Реляционные базы данных плохо адаптируемы к сетям распределенных вычислений, плохо поддерживают объем данных больше 100Тб и накладывают ограничение на жесткую структуру и схему данных.

При работе с большими объемами информации и распределенными системами хранения и обработки информации основными свойствами из теоремы CAP для баз данных являются AP (доступность и устойчивость к расщеплению) или CP (согласованность и устойчивость к расщеплению). Именно базы данных удовлетворяющие свойствам AP и CP называются NoSQL базами данных (Not only SQL). В их основе лежат принципы: хорошая масштабируемость, нереляционная модель данных и зачастую - открытый программный код. Но NoSQL базы данных не освобождены и от недостатков. К ним относятся:

— отсутствие транзакционной логики и контроля целостности в большинстве реализаций, т.е. программисту необходимо реализовывать её в своем приложении самостоятельно. В тоже время специализированная логика может оказаться эффективнее общих алгоритмов реляционных БД;

— для обработки данных необходимо использование дополнительного языка программирования .

На рис.6 приведена классификация NoSQL баз данных по типам.

Ключ-Значение (Key-Value) — базы данных, относящиеся к этому типу представляют собой коллекцию объектов в виде пар "ключ"-"значение", где ключ для каждой пары обязательно уникален. В свою очередь "значение" может быть представлено и другими парами "ключ"-"значение". Таким образом, получается, что глубина вложенности теоретически не ограничена. Основными операциями для таких БД являются операции вставки значения для ключа - insert (key, value), удаление объекта

по ключу - delete (key), обновление значения по ключу - update(key, newValue), проверки наличия значения по ключу -lookup(key) и исполнение операции - execute(key, operation, parameters). В распределенной модели хранения данные распределяются по узлам в соответствии со значением ключа. К "key"-"value" типу баз данных относятся: Oracle NoSQL DataBase, Riak, Dynamo и другие.

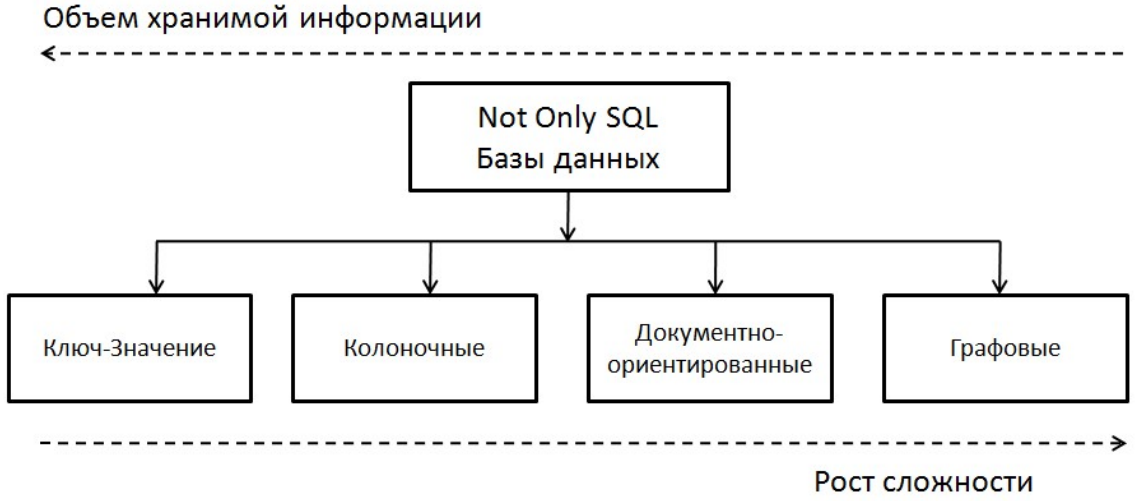


Рис. 6. Классификация NoSQL баз данных по типам.

Колоночные (Wide Column) — этот тип баз данных, который представляет собой хранилище в виде широкой таблицы. Таблица одна, но может содержать большое число колонок и строк, при этом количество колонок для каждой строки может меняться. Колонки объединяются в семейства. Примерами такого типа баз данным могу быть: BigTable, Cassandra, HBase и другие.

Документо-ориентированные (Document Oriented) — этот тип баз данных напоминает тип "ключ"-"значение" базы, но в данном случае, база данных знает, что из себя представляют значения. Обычно, значением является некоторый документ или объект, к структуре которого можно делать запросы. Основное понятие документа – слова, фразы, предложения, параграфы и т.д.. Отличительной особенностью данного типа является гибкая структура – подкомпоненты структуры могут быть вложенными и различной формы. Присутствует поддержка метаданных для документа. Например, название, автор, время, встроенные теги. Примерами форматов

документов могут быть: PDF, XML, JSON, текст, сканированные рисунки и др.. Типичными представителями этого класса являются: MongoDB, ElasticSearch, Cassandra, CouchDB и др.

Графовые (Graph) — эти базы данных ориентированы на поддержку сложных взаимосвязей между объектами, и основываются на теории графов. Структура данных в графовых базах представляет собой набор узлов, связанных между собой ссылками. При этом и узлы и ссылки могут обладать некоторым количеством атрибутов. Основной моделью данных является направленные графы, узлы и грани которых содержат свойства объектов. К таком типу NoSQL БД относятся: Neo4j, IBM System G, GraphBase и другие.

В курсовом проекте предполагается использование одной из распределенных баз данных, относящихся к типу NoSQL. В некоторых случаях может представиться, что использование БД не является рациональным с точки зрения архитектуры системы, но важным является именно получение навыков практической работы с распределенными хранилищами.

На выбор предлагается использовать одно из следующих хранилищ: MongoDB[1], Apache Geode[2], HDFS[3], Redis[4], Cassandra[5], Neo4j[6] и др.

Важным является подчеркнуть целесообразность использования БД в проекте на этапе обработки, в том числе и баз данных, которые хранят свою информацию в памяти (in-memory). За счет использования этого способа хранения значительно повышается скорость вычислений, особенно в режиме обработки в реальном времени.

---

1 MongoBD. [Электронный ресурс]. Режим доступа: // https://www.mongodb.com/

[2] Apache Geode. [Электронный ресурс]. Режим доступа: https://geode.apache.org

[3] HDFS. [Электронный ресурс]. Режим доступа: https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html

[4] Redis. [Электронный ресурс]. Режим доступа: // http://redis.io/

[5] Cassandra. [Электронный ресурс]. Режим доступа: // https://cassandra.apache.org/

[6] Neo4j. [Электронный ресурс]. Режим доступа: https://neo4j.com/

Приветствуется, но не является обязательным, использование Amazon Web Services[1] (AWS) для хранения и обработки данных. AWS предоставляет возможность бесплатного годового сервиса с ограниченной функциональностью, но достаточной для выполнения курсового проекта.

Рекомендуется использование инструмента ElasticSearch в связке с LogStash и Kibana. Эту связку принято называть ELK[2].

5. ОБРАБОТКА ДАННЫХ

Модуль обработки данных является одним из центральных модулей программной системы анализа. В нем заложена основная логика получения ответа на поставленную задачу анализа.

Выбор используемых технологий и методов решения зависит именно от задачи. Для одного типа задач хорошо подходит использование методов машинного обучения и нейронных сетей, а для другого типа задач идеальным образом становится простое решение с помощью SQL-подобных запросов. Таким образом, выбор того, какую технологию использовать, ложится на студента.

На рис.7 приведено развитие основных способов обработки информации и инструменты, используемые для каждого из способов, начиная с пакетной обработки данных в 2000 году и заканчивая обработкой в режиме реального времени. Представленная классификация направлена на облегчение исполнителю выбора технологии обработки. Пакетная обработка (Batch) — способ обработки входного массива информации целиком. На вход обработчику подаётся массив с входными данными, указывается функция, которая обрабатывает каждый элемент массива и число - количество элементов, обрабатываемых за один вызов функцией. Также задаётся функция, вызываемая после завершения обработки всего массива. Обработка данных только на одном вычислительном узле (процессоре или компьютере) ограничивает скорость вычислений и вызывает потерю результатов промежуточной обработки данных в случае отказа этого узла. Именно

[1] Amazon Web Services. [Электронный ресурс]. Режим доступа: // https://aws.amazon.com/ru/

[2] Стек технологий для ElasticSearch. [Электронный ресурс]. Режим доступа: *https://www.elastic.co/webinars/introduction-elk-stack*

поэтому с 2000-х годов начинают свое развитие распределенные файловые системы (DFS) и распределенные способы пакетной обработки данных. Такой шаг позволил обеспечить отказоустойчивость и горизонтальное масштабируемое хранение данных среди вычислительных устройств (узлов). Стоит отметить, что каждый узел имеют свою локальную память, но виртуально все узлы объединяются в единую систему. Сочетание DFS и параллельных вычислений является оптимальным для обрабатывающих приложений с последовательным доступом к данным, работающим с данными по принципу: "один раз запиши, много раз прочитай" (“write once read many”). Ярким примером пакетной обработки является парадигма MapReduce, которая впервые была разработана компанией Google.

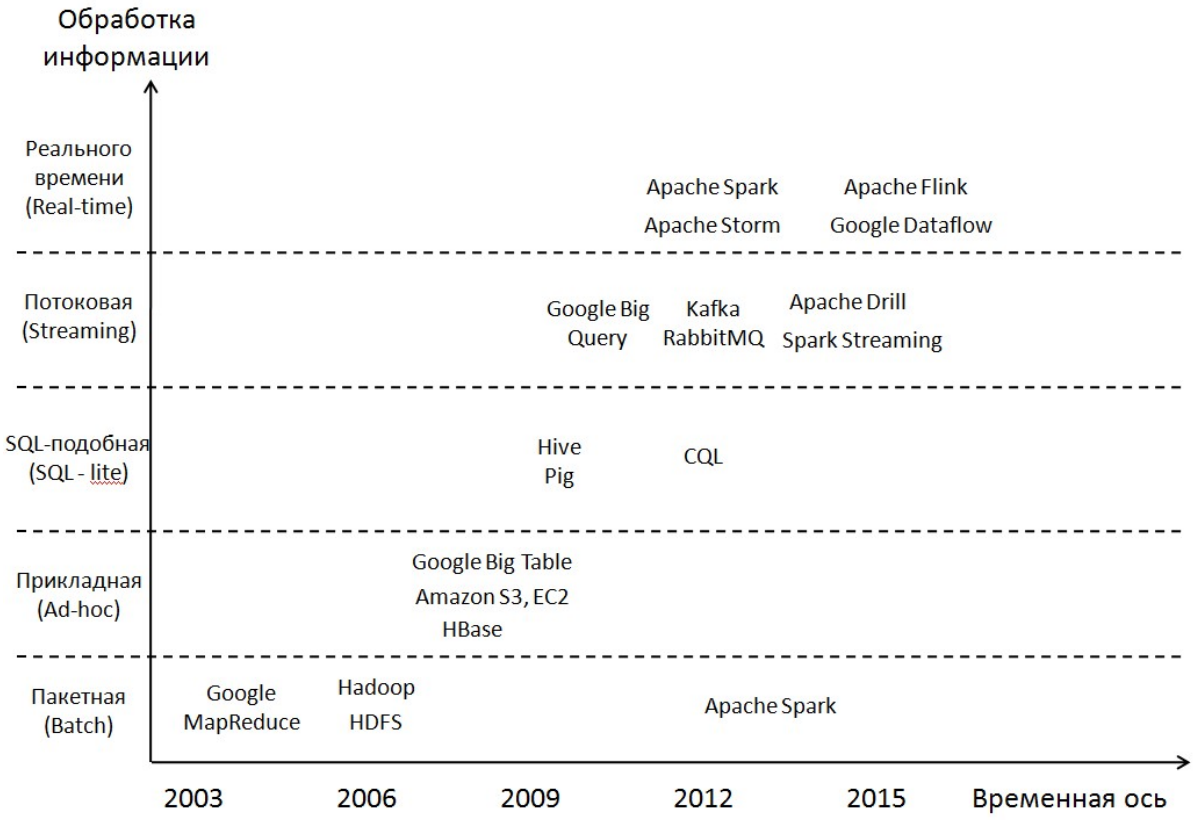


Рис. 7. Способы обработки информации и основные инструменты

Прикладная обработка (Ad-hoc) — это быстрый способ обработки данных, на лету, который требует нерегламентированных запросов и обращений к данным в заданном диапазоне, т.е. произвольный доступ на за-

пись и чтение. Решая задачу поддержки произвольного доступа в распределенных файловых системах и сохраняя возможность проведения пакетной обработки, были разработаны первые NoSQL базы данных. Традиционными NoSQL базами данных являются: HBase, MongoDB, Cassandra.

SQL-подобная обработка (SQL-lite) — это способ обработки информации с помощью запросов, построенных на базе ограниченного SQL-языка. Почти все NoSQL базы стали предоставлять SQL-подобный интерфейс взаимодействия, например, CQL в Cassandra. Способности SQL-подобных интерфейсов и NoSQL баз данных сортировать и организовывать информацию позволили аккумулировать данные в одном месте с последовательным автономным их анализом. Отличительными особенностями SQL-подобных языков является отсутствие поддержки или поддержка с существенными ограничениями операций JOIN и GROUP BY. Применение этих конструкций было ограничено в NoSQL базах данных, потому что сложность вычислений с их использованием растет нелинейно, что пагубно сказывается на производительности.

Потоковая обработка (Streaming) — это способ обработки информации, поступающей в виде потока и генерируемой с какой-либо частотой. Зачастую потоковая обработка информации используется в совокупности с брокерами сообщений, такими как Apache Kafka, Flume или RabbitMQ. В тоже время появляются автономные системы для потоковой обработки, такие как Apache Spark Streaming.

В таблице 1 приведена сравнительная характеристика существующих систем потоковой обработки информации по следующим критериям:

- номер версии;
- дата последнего обновления;
- и тип поддерживаемой обработки.

Целью сравнительного анализа является выбор наиболее перспективной технологии для использования в проекте.

В представленной таблице присутствуют как молодые проекты, так и давно не обновляющиеся, и не широко используемые. Предлагается не рассматривать их детально, но все же привести в таблице для демонстра-

ции широты и многообразия реализаций. Остальных рассматриваемых кандидатов можно разбить на две группы.

Таблица 1. Сравнительный анализ технологий потоковой обработки данных

Название	Номер версии	Дата выхода	Тип обработки данных
Apache Storm	1.0.0	2016-04	stream
Apache Storm Trident	1.0.0	2016-04	micro-batch
Apache Spark Streaming	1.6.1	2016-02	micro-batch
Apache Flink	1.0.1	2016-03	unified stream & batch
Apache Samza	0.10.0	2015-12	stream
Apache Apex	3.3.0-incubating	2016-01	unified stream & batch
S4	0.6.0-incubating	2013-06	stream
Spring XD	1.3.1	2016-02	unified stream & batch
Spring Cloud Data Flow	1.0.0	2015-12	unified stream & batch
Heron	0.14.1 beta	2016-07-06	stream

Группа 1. Storm, Spark и Flink. Представленные технологии используют комплексный подход, в котором фреймворк не только обеспечивает состав потока, но также позволяет управлять потоком, поддерживает высокую доступность данных.

Группа 2. В данной группе технологий, присутствует всего один фреймворк Spring Cloud Data Flow. Он использует подход микросервисов, где сервер потока данных фокусируется на составе потока, а среда выполнения, такая как Cloud Foundry, Apache Mesos или Kubernetes, обеспечивает развертывание и высокую доступность.

Кандидаты в первой группе очень похожи. По функционалу и характеристикам Spark и Flink не сильно отличаются от Storm. Поэтому предлагается сконцентрировать вниманием сравнению Apache Storm и Spring Cloud Data Flow (SCDF).

Результат исследования представлен в таблице 2.

Таблица 2. Сравнение фреймворка Apache Storm и Spring Cloud Data Flow.

Критерий	Apache Storm	Spring Cloud Data Flow
Эксплуатационная надежность (Operability)	Очень ограниченное время использования в отрасли разработки ПО.	Поддерживаются различные широко используемые среды выполнения: Cloud Foundry, Kubernetes, Mesos.
Поддерживаемость (Supportability)	Журналы (logs) разбросаны по рабочим узлам. Предполагается, что Storm UI является центральным местом для поиска журналов, но, к сожалению, это совершенно не работает.	Сводное логгирование обеспечивается средой выполнения. Например, Cloud Foundry имеет компонент loggregator.
Возможность обновления (Upgradability)	Топология должна представлять собой единственный jar, содержащий в себе весь код и все зависимости, но исключая сам Storm. Изменение топологического компонента означает повторное развертывание всей топологии. В следствие чего будет происходить вынужденное бездействие, что весьма нерационально. Даже изменение конфигурации означает перестроение jar топологии.	Каждый компонент топологии представляет собой отдельный микросервис, построенный с использованием Spring Boot. Dataflow Server разворачивает компоненты в среде выполнения. Например, cf push для Cloud Foundry. Изменение топологического компонента означает переразвертывание только этого микросервиса. Конфигурация может быть предоставлена статически с использованием property-файлов или динамически с использованием сервера конфигурации.
Масштабируемость (Scalability)	Максимальное количество экземпляров жестко закодировано в jar топологии. Команда перебалансировки может масштабироваться только в этих пределах.	Предоставляется в режиме runtime. Например, cf scale для CloudFoundry.
Функциональность (Functionality)	Поддержка временных окон.	Документация вкратце упоминает временные окна, но не вдается в подробности о том, как работать с ними. Однако эту функцию легко реализовать с помощью таймеров.
Время разработки (Development time)	Ограниченное количество стандартных компонентов (spouts и bolts). Нет ни одного, что можно было бы использовать.	На GitHub выложены типовые примеры и различные стандартные компоненты.

Из таблицы сравнения по представленным критериям можно сделать вывод, что Spring Cloud Data Flow является наиболее надежным, расширяемым и масштабируемым решением.

Обработка в режиме реального времени (Real-time) — это способ обработки информации, который позволяет предоставлять результаты с минимальной задержкой для пользователя. Несмотря на то, что Apache Spark используется для пакетной обработки данных, эта технология является первой крупной платформой для потоковой обработки данных в реальном времени. И в то же время позволяет использовать сложные алгоритмы машинного обучения.

Ниже даны основные рекомендации, которых следует придерживаться при определении технологий и методов обработки данных.

В случае, когда задача требует пакетной обработки данных, могут использоваться Apache Spark и Apache Hadoop, т. к. они позволяют реализовать обработку с использованием парадигмы MapReduce. В этом случае данные извлекаются из хранилища и обрабатываются большими порциями параллельно на разных вычислительных узлах системы, после чего результаты вычислений на каждом узле объединяются.

При обработке информации в режиме реального времени выбор может быть сделан в пользу Apache Storm[1] с созданием топологии исполнения и/или Apache Spark с использованием модуля Streaming.

При выполнении исследовательских задач может возникнуть необходимость использования методов машинного обучения без учителя, а именно, «метод Тьюки» (метод квантильного расстояния), кластеризация методов К-средних (K-means) и С-средних (C-means), регрессия, а также наивный байесовский классификатор.

Если задача требует использования методов машинного обучения с учителем, рекомендуется использовать следующие методы: дерево принятия решений, «Случайный лес» и другие.

Из технических аспектов реализации рекомендуется использование методов машинного обучения из библиотеки «Deep-learning», в которую

[1] Apache Storm. [Электронный ресурс]. Режим доступа: https://storm.apache.org/

входят перечисленные выше алгоритмы в рамках реализации с использованием фреймворка Spark. Также рекомендуется использование языка R и его пакетов, поскольку он предоставляет широкий спектр реализации готовых методов.

Для работы и анализа неструктурированного текста рекомендуется использовать поисковые механизмы с обратным индексом, который хранится в облачной инфраструктуре. К реализациям поисковых библиотек и фреймворков можно отнести Lucene[2], Solr[3], ElasticSearch[4]. При необходимости производить семантический анализ текста, рекомендуется применение алгоритмов Word2Vec и Glove.

Важным составляющим элементом представления результатов обработки являются временные характеристики разработанной программной реализации. Для всех задач необходимо отразить входной объем данных, поступивший в обработку, время обработки и вычислить скорость обработки. Результаты следует изобразить в таблице с указанием размерностей измеряемых величин.

При реализации модуля обработки рекомендуется обратить особое внимание на производительность решения, на возможность масштабирования решения, на утечки памяти и места, которые возможно оптимизировать.

5.1. ПОДХОД РЕШЕНИЯ ПРОГНОСТИЧЕСКОЙ ЗАДАЧИ

Рассмотрим подход к решению прогностической задачи на основе известных исторических данных, изображенный на рис. 8.

Задача сводится к разработке алгоритма, который с определенной точностью предсказывал бы значение выбранного атрибута объекта в будущем. Для данной задачи актуальны два вопроса. Первое: как доказать, что алгоритм работает корректно? Второе: как доказать, что алгоритм обладает высокой точностью?

---

[2] Lucene. [Электронный ресурс]. Режим доступа: http://lucene.apache.org/core/

[3] Solr. [Электронный ресурс]. Режим доступа: http://lucene.apache.org/solr/

[4] Elasticsearch. [Электронный ресурс]. Режим доступа: https://www.elastic.co/products/elasticsearch

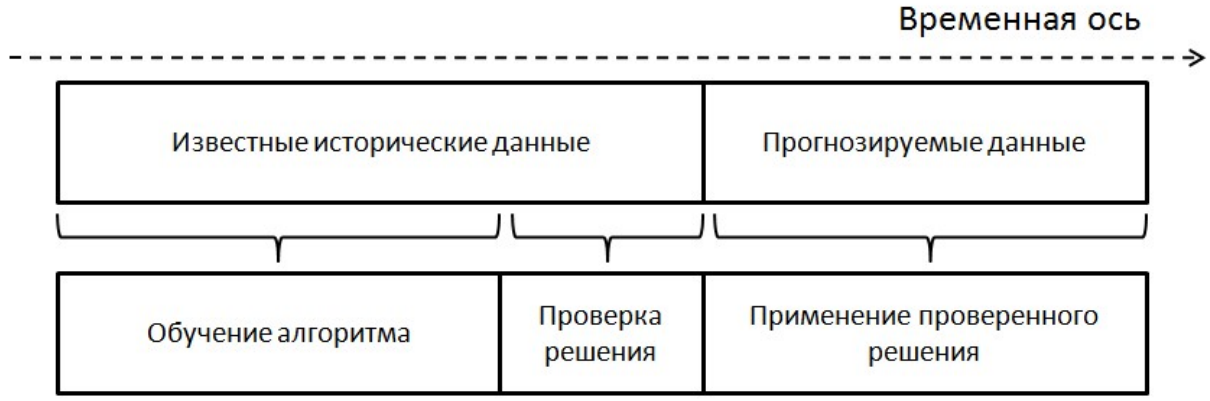


Рис. 8. Подход к реализации прогностической задачи

Чтобы обеспечить высокую точность и возможность доказательства корректности работы алгоритма пытаются предсказать уже известные данные. Исторические известные данные разбивают на два множества: данные на основе которых будет делаться предсказание и данные, которые будут предсказывать. Затем реализуют алгоритм и проверяют, насколько предсказание отклоняется от тех, известных, данных, которые требовалось получить. Если установленная погрешность соблюдается, то алгоритм применяют на всех исторических данных для того, чтобы заглянуть в будущее.

Прогностические задачи в основном решаются за счет применения методов машинного обучения, как с учителем, так и без учителя.

Хорошим примером решения такой задачи является задача прогнозирования заработной платы в IT сфере по данным с HeadHunter.

5.2. ПРИМЕР РЕШЕНИЯ ЗАДАЧИ ПОИСКА НЕКОРРЕКТНЫХ И ОШИБОЧНЫХ ДАННЫХ

На сегодняшний день практически все компании и организации уделяют особое внимание устранению уязвимостей в своих системах. Эти уязвимости могут быть, как в используемом программном обеспечении в виде ошибок, отсутствия модулей шифрования и аутентификации, так и в данных, которыми оперируют программы. В таких сферах деятельности, как финансы, банки, медицина важно в минимально короткое время опре-

делить наличие, например, аномальных финансовых транзакций, или неправильно назначенное лечение. Также, например, в области образования и науки важным является проверка соответствия темы статьи, дипломной работы, его содержимому. Все перечисленные примеры относятся к задачам по выявлению ошибочных или некорректных данных.

Существует достаточное количество инструментов и систем, которые позволяют автоматически выявлять подозрительные данные. К ним относятся: продукт Statistica компании statsoft[5]; продукт Simility[6]; Neo4j - графовые базы данные, на которых базируются системы определения уязвимостей[7]; CPA detective[8] - система определения уязвимостей, предлагаемая для использования маркетинговыми компаниями; Software AG Banking: Fraud detection[9] - система, работающая с численными данными и датами, направленная для применение финансовыми компаниями; и др.. В основе логики этих инструментов заложены принципы машинного обучения, которые направлены на то, чтобы обучить на готовом, существующем, наборе данных программу классифицировать входные данные на два сектора: подозрительные или корректные. Отличительной особенностью этих подходов и инструментов является то, что их логика в основном основана на численных данных. Но в тоже время, подозрительные данные могут быть представлены и неформализованным текстом.

Рассмотрим пример решения, который позволяет автоматически определять подозрительный неформализованный текст на естественном языке. Под подозрительностью текста будем понимать его отношение к определенной тематике. Например, текст, написанный на тему ядерной физики может быть запрещен к упоминаниям во внешней переписке в государственных компаниях. Тематика теста может задаваться не просто од-

---

[5] STATISTICA. [Электронный ресурс]. Режим доступа: http://www.statsoft.com/Products/STATISTICA-Features

[6] Simility. [Электронный ресурс]. Режим доступа: https://simility.com

[7] Gorka Sadowksi, Philip Rathle Fraud Detection: Discovering Connections with Graph Databases - white paper - 2015 год

[8] CPA detective [Электронный ресурс]. Режим доступа: http://cpadetective.com/technology.html

[9] Banking fraud detection [Электронный ресурс]. Режим доступа: http://softwareag.com/us/solutions/banking/fraud_detection/overview/default.asp

ним словом, описывающим определенный раздел, но и целым набором предложений, как, например, заголовок письма или аннотация. Таким образом, для решения поставленной задачи требуется разработать приложение, которое на вход принимает два входных параметра:

-неформализованный текст;

-тематика текста.

Для реализации данной задачи необходим инструмент, который позволяет преобразовывать предложения естественного языка в численные значения, для дальнейшей обработки математическими методами. В работе предлагается использовать алгоритм Word2Vec[10], который эффективно преобразует представления слов в векторное пространство.

Word2Vec – это набор алгоритмов обработки естественного языка, использующий двуслойные нейронные сети для векторного представления слов. Используя его, разработанное решение позволяет определить, соотносится ли входное предложение по смыслу со входным текстом. Отличительной особенностью Word2Vec является наличие готовой языковой модели, которая содержит информацию о том, какие слова есть в языке и как они соотносятся друг с другом. Готовые, обученные модели находятся в открытом доступе и доступны для широкого спектра языковых корпусов. При необходимости, можно обучить модель самостоятельно на текстах из прикладной предметной области.

На рис.9 представлена модульная схема решения для поиска корреляции между входным текстом и тематикой (на схема изображено как «входное предложение»).

[10] Mikolov, Tomas; et al. "Efficient Estimation of Word Representations in Vector Space», arXiv:1301.3781

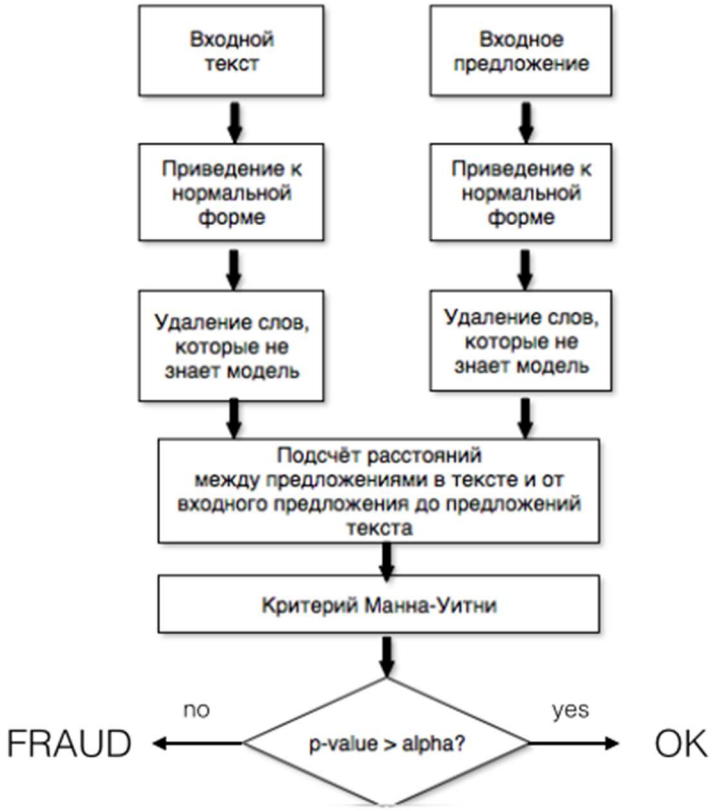


Рис. 9. Алгоритм поиска корреляции между входным текстом и тематикой

Вначале входное предложение и входной текст преобразуются в нормальную форму (например, «собаки» -> «собака», «сделал» -> «сделать»). Для этого используется сторонний пакет jmorphy2[11], java-порт pymorphy2[12]. pymorphy2 - инструмент, позволяющий приводить слова к нормальной форме, ставить слово в нужную форму (менять падеж, число), возвращать грамматическую информацию о слове.

Затем, слова из нормализованного текста которых нет в языковой модели, и предложения, в которых все слова отсутствуют в языковой модели (например, предложения, написанные на другом языке, предложения, состоящие из малоизвестных слов, бессмыслица), удаляются. Далее, каждое из слов приводится к векторной форме с помощью алгоритма

[11] Github/Jmorphy2. [Электронный ресурс]. Режим доступа: https://github.com/anti-social/jmorphy2

[12] Korobov M.: Morphological Analyzer and Generator for Russian and Ukrainian Languages // Analysis of Images, Social Networks and Texts, pp 320-332 (2015)

word2vec. Word2Vec обучается так, что слова, близкие по контексту, имеют высокую косинусную близость. Сумма векторов слов даёт вектор слова, несущего «общий смысл». Например, «король» + «женщина» + «ребёнок» = «принцесса». Это свойство используется для решения поставленной задачи.

Для каждого предложения полученные вектора суммируются, что даёт общий вектор предложения, показывающий итоговый смысл. Далее, составляются две выборки, первая выборка содержит косинусные расстояния между каждым и каждым предложениями текста, а вторая выборка - косинусные расстояния между входным предложением и каждым предложением входного текста. Затем к полученным выборкам применяется критерий Манна-Уитни[13]. Критерий Манна-Уитни используется для оценки различий между двумя независимыми выборками по уровню какого-либо признака, измеренного количественно, а также позволяет выявлять различия в значении параметра между малыми выборками. В данном случае между значениями смыслового отклонения предложений текста друг от друга и значениями отклонения входного предложения от предложений текста. В случае, если критерий возвращает значение p-value, меньшее заданного заранее пользователем значения alpha, алгоритм отвергает гипотезу о равенстве значений, тем самым утверждая, что входной текст и входное предложение по смыслу отличаются.

Представленный выше алгоритм реализован в библиотеке[14], позволяющей определять, соответствует ли по смыслу входное предложение входному тексту. Программным интерфейсом библиотеки является класс JSentenceDetection с единственным публичным методом detect. На вход этот метод принимает предложение и текст, и возвращает логическое значение, показывающее, соответствует ли смысл текста и предложения.

---

[13] Mann H. B., Whitney D. R. On a test of whether one of two random variables is stochastically larger than the other. // Annals of Mathematical Statistics. — 1947. — № 18. — P. 50—60.

[14] Github/JSentenceDetection. [Электронный ресурс]. Режим доступа: https://github.com/nosov-pvl/jsentencedetection

Рассмотрим несколько примеров применения решения. Используем в качестве входного текста статью о влиянии кофе на артериальное давление, а в качестве входного предложения: «У людей, страдающих гипертонией, давление может резко и сильно вырасти до критических значений, угрожающих здоровью».

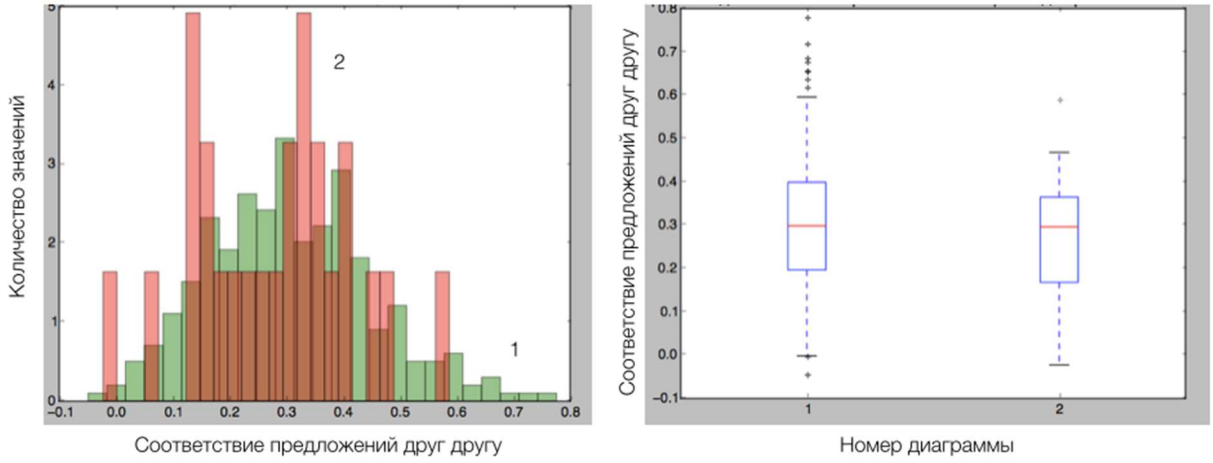


Рис. 10. Результат работы программы для примера соответствия
(а) гистограмма соответствия (б) диаграмма размаха

После применения программного решения получены результаты, представленные на Рис.10 (а, б). На рис 9.а красная гистограмма относится к распределению значений соответствия входного предложения предложениям из текста, а зеленая к распределению значений соответствия предложений текста друг другу. На рис. 9.б левая диаграмма размаха, показывает распределение значений соответствия предложений текста друг другу, а правая - распределение значений соответствия входного предложения предложениям из текста. Из гистограмм и диаграмм размаха видно, что предложение соответствует тексту по смыслу. Что подтверждает и результат работы теста и значение p-value равное 0.454577.

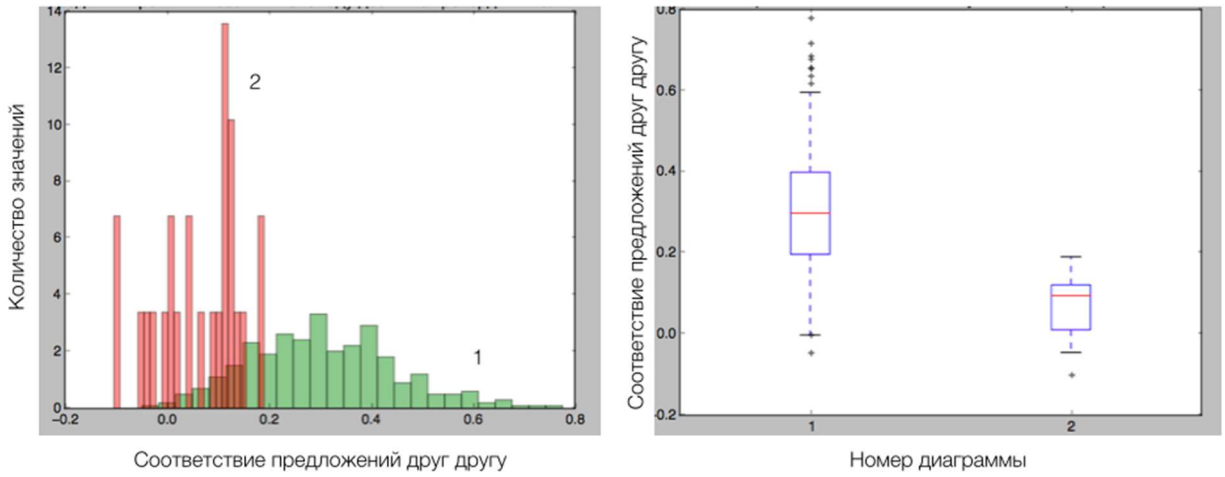


Рис. 11. Результат работы программы для примера несоответствия
(а) гистограмма соответствия (б) диаграмма размаха

Рассмотрим входное предложение «Кредитный рейтинг России в 2016 году достигнет рекордных величин.», оставив входной текст неизменным.

Из графиков, представленных на рис.11 (а, б) видно смысловое несоответствие предложения и текста, что подтверждает и результат работы теста, и значение p-value равное 0.

6. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ

Результаты проведенных исследований над данными должны быть представлены в виде набора графиков и диаграмм, наглядно изображающих полученные выводы. Возможно использование подходов изображения результатов в виде инфографики или облака тегов.

Для визуализации результатов аналитики разумно применение языка R[15]. Он может быть использован для обработки «сырых» результатов анализа, объем которых не превышает 200…300 Мб, так как программы, созданные на нем (языке R), являются относительно медленными и не масштабируемыми, хотя на сегодняшний день и есть поддержка языка R в Spark для исполнения на кластере. Программы на языке R могут быть использованы для быстрого и наглядного изображения промежуточных ре-

[15] Язык R. [Электронный ресурс]. Режим доступа: https://www.r-project.org/

зультатов, на основе которых делается выбор дальнейшего направления движения при ответе на поставленный в задаче вопрос.

Для визуализации возможно использование инструмента Kibana или Banana, которые позволяют формировать запросы к готовым данным, хранящимся в архиве и заранее подготовленным для запросов данным.

Один из возможных сценариев использования визуализации с применение инструмента Kibana[16] представлен на рис.12.

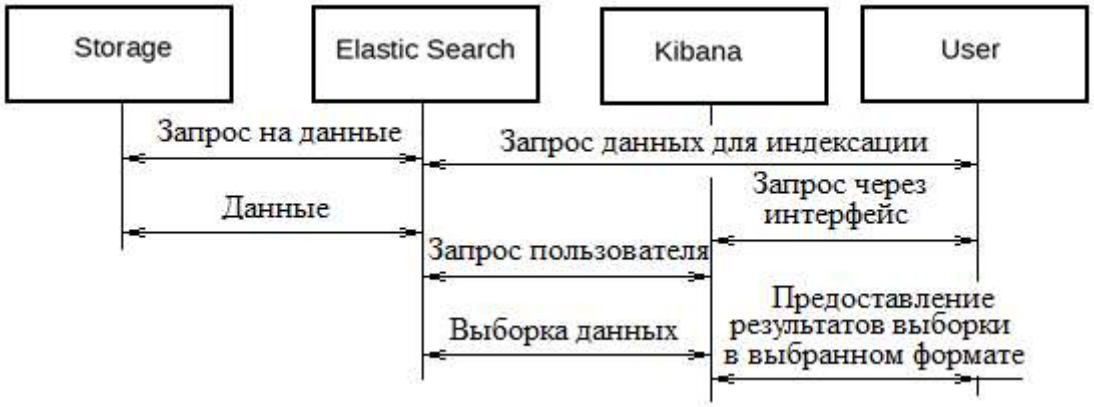


Рис. 12. Сценарий использования инструмента Kibana для визуализации результатов аналитики

Данные, приведенные в таблицах, а также графики обязательно комментируются в текстовой форме.

Каждая таблица должна иметь название. Оси каждого из графиков должны быть подписаны. Для каждого графика или диаграммы должны быть указаны численные характеристики входных данных. Для каждого графика должны присутствовать выводы, которые можно сделать исходя из одного графика или совокупности графиков. Удовлетворение этих требований позволяет эффективно и наглядно изобразить результаты проделанной работы в отчете и в презентации.

Для тех, кто выберет языком реализации Python, можно использовать различные библиотеки визуализации и построения графиков. Например, iGraph.

[16] Kibana. [Электронный ресурс]. Режим доступа: www.elastic.co/products/kibana

7. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Программная реализация курсового проекта может быть представлена в виде самостоятельного (standalone desktop) приложения, реализованного на любом языке программирования, в котором студент чувствует себя свободно. Но стоит отметить, что вряд ли успешный проект по аналитике будет представлен одним приложением и написан на только одном языке программирования.

Успешные и, в той или иной мере, универсальные приложения, представлены в виде web-application с наличием front-end и back-end частей, взаимодействие между которыми осуществляется по принципам REST-сервисов.

Для выполнения курсового проекта рекомендуется, но не обязательно, использовать следующие языки программирования: Java, C#, Python, R, SQL, JavaScript.

Таким образом, например, модуль выгрузки данных может быть написан на языке Python, модуль приема и хранения данных на языке C#, модуль обработки данных на языке Java, модуль визуализации результатов на языке R или JavaScript.

При выполнении работ над проектом рекомендуется использование системы контроля версий, например, Git (BitBucket или GitHub), в котором можно отслеживать процесс выполнения студентом хода работ над проектом.

8. ЗАЩИТА ПРОЕКТА

Курсовой проект предполагает защиту, по результатам которой выставляется итоговая оценка. По положению защита проекта проводиться в комиссии, которую формирует заведующий кафедрой. Защита проводится в последнюю неделю семестра в виде презентации с демонстрацией результатов. Предполагается, что студенты заранее выберут удобное время выступления и запишутся в список, сформированный старостами групп. Старосты групп должны передать график выступающих студентов преподавателю не позднее середины семестра.

Пример графика защит представлен в таблице 3.

Таблица 3. Пример графика защит

17.12.2016	
10-00	Иванов Петр гр. 13504/1
10-15	Курочкин Иван гр. 13504/2
10-30	Сидоров Михаил гр. 13504/3
…	…..
13-45	Петров Петр гр. 13504/2

24.12.2016	
10-00	Петров Иван гр. 13504/1
10-15	Васильев Андрей гр. 13504/2
10-30	Толстой Лев гр. 13504/3
…	…..
13-45	Бунин Иван гр. 13504/2

Выступление рассчитывается на презентацию порядка 20 слайдов и порядка 10…15 минут по времени. В своем выступлении студент обосновывает актуальность темы, ее практическое значение, кратко излагает основные вопросы и результаты выполненных исследований.

После выступления студента преподаватели задают ему вопросы по теме работы. На вопросы студент должен отвечать кратко. Полнота и глубина ответа влияют на общую оценку.

Если по сформулированным исполнителем задачам исследования в открытом доступе существуют сторонние результаты, то необходимо их привести, сослаться и использовать для проверки собственного решения.

Рекомендации к оформлению и содержанию слайдов презентации представлены ниже:

— рекомендуется в начале презентации расположить слайд с описанием темы, предметной области и API источника данных. Отсутствие данного слайда может привести к утрате логики выступления;

— требуется разместить отдельный слайд, на котором необходимо перечислить сформулированные задачи для анализа;

— предполагается наличие слайда с информацией о том, какой объем информации был обработан, какое количество объектов обработано, какое количество времени было потрачено на получение информации, на обработку, визуализацию и осмысление. То есть необходимо привести сухую статистику;

— представленные графики, диаграммы и гистограммы должны быть подписаны, все шкалы должны иметь названия и размерности. Все подписи должны быть читаемыми. Для тех, кто использует для визуализации Kibana, рекомендуется точно откалибровать шрифты подписей для

осей. Также рекомендуется задумываться над тем, как презентация выглядит не на мониторе компьютера или ноутбука, а на проекторе;

— все графики и диаграммы должны иметь сопровождение в виде выводов и тезисов;

— презентация должна быть богата на технические детали, поскольку исполнитель должен продемонстрировать квалификацию инженера.

— все слайды, за исключением титульного, должны быть пронумерованы;

— одним из последних слайдов должен быть слайд со списком использованной литературы и ссылкой на репозиторий проекта в системе контроля версий.

Затем, после успешной презентации студент сдает отчет о проделанной работе. На титульном листе обязательно должны присутствовать фамилия, имя и отчество, номер группы, учебный год и подпись студента. Пример титульного листа приведен в «Приложении 1». В отчете должны быть отражены следующие моменты:

— описание предметной области, которое включает: теоретическое и практическое значение выбранной темы, ее актуальность, степень разработанности проблемы;

— описание конкретных задач исследования (в виде проработанных вариантов использования, Use Cases);

— описание модели данных (в виде UML-диаграмм);

— используемые алгоритмы, техники и подходы анализа;

— графики и диаграммы;

— характеристики разработанной системы;

— выводы;

— используемая литература.

Введение должно быть кратким по содержанию (1…2 страницы) и четким по стилю изложения. Его не следует перегружать общими фразами. Главное, чтобы читающий понял, чему посвящена работа, какие задачи автор сам для себя наметил.

Основная часть (7…10 страниц) состоит из глав, которые могут делиться на параграфы, а параграфы, в свою очередь — на пункты. Название какой-либо главы не должно полностью совпадать с названием курсовой работы (в противном случае наличие других глав становится излишним), а название какого-то параграфа не должно дублировать название главы. Основная часть работы содержит технические аспекты реализации программного продукта, направленного на решение поставленной задачи. В отчете не допускается вставка программного кода, вместо этого рекомендуется помещать модульные схемы и алгоритмы.

В итоговом отчете в разделе «Заключение» (объем не больше 1 страницы) следует четко сформулировать основные выводы, к которым пришел автор. Выводы должны быть краткими и органически вытекать из содержания работы. Разрешается повторить основные выводы соответствующих глав, но при этом предпочтительнее стремиться сделать некоторые обобщения по результатам проведенного исследования в целом.

Список использованной литературы (не менее 5 источников) оформляется по установленному порядку. Он включает в себя всю литературу, на которую есть ссылки в тексте, а также те важнейшие источники, которые были так или иначе использованы, хотя и не приведены в ссылках и примечаниях.

В количестве приложений в отчете исполнитель не ограничен.

9. ПРАВИЛА ФОРМИРОВАНИЯ ИТОГОВОЙ ОЦЕНКИ

Итоговая оценка по курсовому проекту выставляется на основе следующих критериев:

— соответствие объема исходных данных для проведения анализа установленным требованиям в разделе «Формирование набора данных»;

— соответствие количества сформулированных задач анализа и их сложность установленным нормам, приведенным в разделе «Формирование набора данных»;

— использование продвинутых методов аналитики;

— использование технологий обработки данных с помощью фреймворков из экосистемы Hadoop;

— полнота и оформление представленного отчета;

— сроки выполнения работы.

Стоит отметить, что при выборе курсового проекта «Использование существующих решений и наборов данных» итоговая оценка будет не высокой, т. к. трудозатраты на воспроизведение готового решения минимальны.

В случае, если объем данных не будет превышать установленных норм, то оценка за курсовой проект не будет превышать «хорошо».

При несоблюдении установленных сроков выполнения работы и сдачи отчета по курсовому проекту по неуважительной причине — итоговая оценка не будет превышать «удовлетворительно».

Программные реализации для наиболее успешных курсовых проектов можно установить на кафедральном кластере и осуществить тестирование их работоспособности в распределенном режиме, в том числе установить предельные величины нагрузки и провести масштабирование системы.

УЧЕБНАЯ ЛИТЕРАТУРА

1. Аверьянов Л. Я. Контент-анализ. — М.: КноРус, 2009. — 451 с.

2. Барсегян А. А. Анализ данных и процессов. — СПб.: Изд-во БХВ-Петербург, 2009. — 512 с..

3. Карау Х., Конвински Э. Изучаем Spark. Молниеносный анализ данных. — ДМК Пресс, 2015. — 304 с.

4. Смородин В. В., Волкова Е. В., Алиев А. А. От хранения данных к управлению информацией. — СПб.: Изд-во Питер, 2010. — 528 с.

5. Фаулер М., Прамодкумар Дж. Садаладж. NoSQL: новая методология разработки нереляционных баз данных. — М.: «Вильямс», 2013. — 192 с.

6. Ryza S., Laserson U., Owen S., Wills J., Advanced Analytics with Spark, O’Reilly, 2015 — 276 с.

7. Warder P., Big Data Glossary, O'Reilly, 2011 — 62 с

8. White T., Hadoop: The Definitive Guide. — 2-е изд. — O’Reilly, 2011. — 625 с.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

ОБРАЗЕЦ ОФОРМЛЕНИЯ ТИТУЛЬНОГО ЛИСТА

Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и технологий

Высшая школа программной инженерии

АНАЛИЗ ЖУРНАЛА ЗАПРОСОВ К ЭЛЕКТРОННОЙ РОССИЙСКОЙ НАЦИОНАЛЬНОЙ БИБЛИОТЕКЕ

Курсовой проект
по учебной дисциплине
«Наука о данных и аналитика больших объемов информации»
Направление подготовки магистров 09.04.04 «Программная инженерия»

Группа 13544/__

Студент: _____ _____
(подпись) *(инициалы, фамилия)*

Руководитель:
доцент, к. т. н. _____ _____

Санкт-Петербург
20__

Примечания к оформлению титульного листа:

1. Используется межстрочный интервал с множителем 1,2.

2. Для оформления титульного листа используются гарнитуры шрифтов, указанные в следующей таблице:

Фрагмент титульного листа	Гарнитуры шрифтов		
	Кегль	Начерта-ние	Буквы
Организация, институт, кафедра	14	светлое	строчные
Фамилия, инициалы автора	16	жирное	строчные
Название работы	18	жирное	пропис-ные
Направление подготовки	16	светлое	строчные
Группа, студент, руководитель	14	светлое	строчные
«Санкт-Петербург, 20__»	16	светлое	строчные

ПРИЛОЖЕНИЕ 2

УПРАЖНЕНИЯ ПО ТЕХНОЛОГИИ APACHE HADOOP

ВВЕДЕНИЕ

Цель данного цикла упражнений — получение практических навыков по работе с компонентами технологий, входящих в экосистему фреймворка Apache Hadoop. Выполнение упражнений отдельно от курсового проекта позволяет студенту познакомиться поближе с интересующим его фреймворком.

Hadoop[17] — проект фонда Apache Software Foundation, свободно распространяемый набор утилит, библиотек и фреймворк для разработки и выполнения распределённых программ, работающих на кластерах из нескольких сотен или тысяч узлов. Используется для реализации поисковых и контекстных механизмов многих высоко нагруженных веб-сайтов. Разработан на языке Java в рамках вычислительной парадигмы MapReduce[18], согласно которой приложение разделяется на большое количество одинаковых элементарных заданий, выполнимых на узлах кластера (компьютерах) и естественным образом сводимых в конечный результат.

Основной операционной системой Hadoop является Linux, но возможна установка и на операционной системе Windows. Для общности и простоты разбирательства с Hadoop рекомендуется использовать Linux. Для тех, у кого эта система отсутствует и основная операционная система Windows, предлагается использовать VirtualBox с установленной клиентской ОС Linux или любой другой способ виртуализации.

Для успешного выполнения упражнений необходимо умение пользоваться документацией на используемые технологии.

Для выполнения упражнений необходимо осуществить предварительную конфигурацию рабочего окружения. Ниже перечисленные основные шаги.

[17] Tom White. Hadoop: The Definitive Guide, 4th Edition, O'Reilly Media, 2015.

[18] Thilina Gunarathne. Hadoop MapReduce Cookbook. Hadoop MapReduce v2 Cookbook Second Edition. Packt Publishing, 2015

1. Установите Linux в Virtual Box и настройте общую папку для того, чтобы из домашней операционной системы было проще и привычнее переносить файлы. Рекомендуется использовать Ubuntu не ниже 14.0 версии.

2. Установите дополнения гостевой ОС для Virtual Box.

3. Установите JDK. Минимальная версия Java 1.7, но рекомендуемая Java 1.8 (поддерживается, начиная с версии Hadoop 2.3).

На момент создания упражнений вышла версия Hadoop 3.0[19], но она еще не является стабильной. Исходя из этого рекомендуется использовать версию 2.x.x.

Упражнения предусматривают работу с HBase[20], поэтому при выборе версии рекомендуется опираться на таблицу П2.1 совместимости версий HBase с версиями Hadoop (например, не рекомендуется выбирать версии 2.6.0 и 2.7.0 ввиду отсутствия для них совместимых версий HBase). При составлении таблицы были использованы обозначения: “S” – поддерживается, “NT” – не протестировано, “X” – не поддерживается.

Таблица П2.1. Совместимость версий HBase с версиями Hadoop.

	HBase						
	0.94.x	**0.98.x**	**1.0.x**	**1.1.x**	**1.2.x**	**1.3.x**	**2.0.x**
Hadoop-1.0.x	X	X	X	X	X	X	X
Hadoop-1.1.x	S	NT	X	X	X	X	X
Hadoop-0.23.x	NT	X	X	X	X	X	X
Hadoop-2.2.0	NT	S	NT	NT	X	X	X
Hadoop-2.3.x	NT	S	NT	NT	X	X	X
Hadoop-2.4.x	NT	S	S	S	S	S	X
Hadoop-2.5.x	NT	S	S	S	S	S	X
Hadoop-2.6.0	X	X	X	X	X	X	X
Hadoop-2.6.1 +	NT	NT	NT	NT	S	S	S
Hadoop-2.7.0	X	X	X	X	X	X	X
Hadoop-2.7.1+	NT	NT	NT	NT	S	S	S

[19] Официальный сайт Apache Hadoop, предоставляющий возможность скачать любую версию фреймворка. [Электронный ресурс]. Режим доступа: // http://www.apache.org/dyn/closer.cgi/hadoop/common/

[20] Apache HBase. [Электронный ресурс]. Режим доступа: // https://hbase.apache.org/

УПРАЖНЕНИЕ 1 – УСТАНОВКА HADOOP И РАБОТА С HDFS

Цель данного упражнения — получение практических навыков установки и администрирования фреймворка Apache Hadoop и распределённой системой HDFS[21]. Упражнение затрагивает установку и запуск Hadoop, работу с файловой системой и с веб-интерфейсом фреймворка.

Теоретический материал

Одной из основных и важных характеристик фреймворка Apache Hadoop является работа с распределенной моделью данных и вычислений. Данный фреймворк рассчитан на работу на кластере. На каждом компьютере кластера должно присутствовать две компоненты: Task Tracker[22] и Data Node[23]. Task Tracker — выполняет маленькую вычислительную задачу на узле. Задача Data Node — управлять данными, которые были переданы именно этому узлу. Узлы, которые содержат только две данных компоненты, носят название Slave. Также в системе должен присутствовать Master узел, координирующий работу Slave узлов. Master узел имеет в себе не только компоненты Task Tracker и Data Node, но и Job Tracker[24] и Name Node[25].

Job Tracker и Task Tracker — это части компоненты MapReduce.

Name Node и Data Node — это части компоненты HDFS.

Приложение, которое будет исполняться на Hadoop, будет контактировать только с Master Node. Основная архитектура приложения — это набор процессов.

Роль и цель Job Tracker, который исполняется на Master Node, — разбить большие задачи на маленькие кусочки и отправить каждую ма-

---

[21] Пользовательское руководство по HDFS от Apache. [Электронный ресурс]. Режим доступа: // https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html

[22] Документация Hadoop по TaskTracker. [Электронный ресурс]. Режим доступа: // https://wiki.apache.org/hadoop/TaskTracker

[23] Документация Hadoop по DataNode. [Электронный ресурс]. Режим доступа: // https://wiki.apache.org/hadoop/DataNode

[24] Документация Hadoop по JobTracker. [Электронный ресурс]. Режим доступа: // https://wiki.apache.org/hadoop/JobTracker

[25] Документация Hadoop по NameNode. [Электронный ресурс]. Режим доступа: // https://wiki.apache.org/hadoop/NameNode

ленькую часть вычислений соответствующему Task Tracker на узле. Task Tracker в свою очередь, когда сможет закончить вычисления, отправит обратно информацию в Job Tracker, который объединит все результаты вычислений и отправит их обратно приложению.

Цель Name Node — хранить индекс того, какая часть данных находится на каком узле. Приложение обращается к Name Node и получает от него информацию об узле, где хранится кусок данных и затем обращается к нужному узлу напрямую.

Обобщенная схема архитектуры фреймворка Apache Hadoop представлена на рис. П2.1.

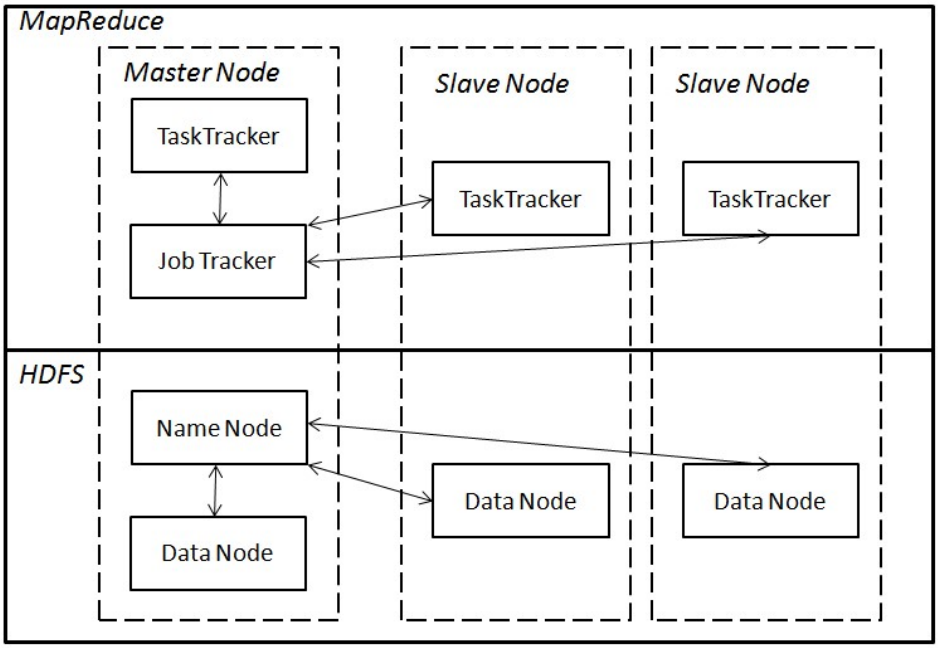


Рис. П2.1. Архитектура фреймворка Apache Hadoop

Существует несколько вариантов использования Hadoop:

— автономный режим (Standalone Mode) используется для того, чтобы на локальной машине применить Hadoop как кластер, состоящий из одного узла. Данные хранятся в локальной файловой системе;

— псевдо распределенный режим (Pseudo Distributed Mode). В данном режиме Hadoop работает в «вырожденном кластере». Все задачи выполняются на одной машине отдельными процессами, для хранения данных используется HDFS, так же используется JobTracker;

— полностью распределенный режим (Fully Distributed Mode) используется на реальном кластере.

Для каждого из вариантов требуется своя последовательность установки и настройки. Для выполнения упражнения предлагается установить Hadoop в псевдораспределённом режиме для возможности работы с HDFS.

Ход выполнения упражнения

1. Установите Apache Hadoop на компьютер в псевдо распределённом режиме. Для работы с Hadoop создайте отдельного пользователя системы. Например, назовите пользователя — hduser.

2. Запустите сервисы Hadoop.

3. Используя команду *jps*[26] проверьте, что успешно запущены все демоны Hadoop: NameNode, DataNode, NodeManager [27] и ResourceManager[28].

4. Изучите команды для работы с HDFS[29]:

— навигация по директориям системы и отображение их содержимого;

— копирование из локальной системы в HDFS и обратно;

— вывод содержимого файла;

— удаление файла/директории;

— и др.

5. Создайте любой текстовый файл в домашней директории, скопируйте его в HDFS (для сдачи упражнения, необходимо показать 2 способа помещения файла в HDFS)

6. Из HDFS выведите в консоль содержимое скопированного файла.

7. Скопируйте файл назад в домашнюю директорию.

---

[26] Документация Oracle по jps – команде, отображающей Java-процессы. [Электронный ресурс]. Режим доступа: // http://docs.oracle.com/javase/7/docs/technotes/tools/share/jps.html

[27] Документация Hadoop по NodeManager. [Электронный ресурс]. Режим доступа: // http://hortonworks.com/blog/apache-hadoop-yarn-nodemanager/,

[28] Документация Hadoop по ResourceManager. [Электронный ресурс]. Режим доступа: // http://hortonworks.com/blog/apache-hadoop-yarn-resourcemanager/

[29] Список команд HDFS. [Электронный ресурс]. Режим доступа: // http://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/FileSystemShell.html

8. Откройте в браузере веб-интерфейс Hadoop, доступный по ссылке: http://localhost:50070 (адрес - по умолчанию), и посмотрите с помощью веб-интерфеса содержимое файловой системы.

Для успешной защиты упражнения требуется:

— запустить Hadoop у себя на компьютере;

— с помощью команды *jps* показать, что демоны (NameNode, DataNode, NodeManager, ResourceManager) успешно запущены;

— выполнить все шаги, связанные с работой с HDFS;

— запустить веб-интерфейс Hadoop и показать в нём созданные в HDFS файлы.

УПРАЖНЕНИЕ 2 – РАЗРАБОТКА MAP-REDUCE ЗАДАЧИ

Цель данного упражнения — получение практических навыков запуска и создания распределенных MapReduce задач. В упражнении будут запущены готовые примеры MapReduce задач, имеющиеся в Hadoop. Также подразумевается, что в данном упражнении будет разработано собственное MapReduce приложение с нуля.

Теоретический материал

Приведем описание этапов обработки данных в Hadoop. Обработка данных подразделяется на этапы, перечисленные ниже.

1. Запуск приложения: передача кода приложения на главный (master) и подчиненные узлы (workers).

2. Мастер назначает конкретные задачи (Map или Reduce) и распределяет части входных данных по вычислительным узлам (workers).

3. Map-узлы читают назначенные им входные данные и начинают их обработку.

4. Map-узлы локально сохраняют промежуточные результаты: каждый узел сохраняет полученный результат на локальные диски.

5. Reduce-узлы, после окончания обработки всех задач на Map-узлах, сортировки результата по ключу, распределяют зачади по ключу и выполняют Reduce обработку данных.

6. Reduce-узлы сохраняют итоговые результаты в выходные файлы, обычно в HDFS.

Схема работы MapReduce представлена рис. П2.2.

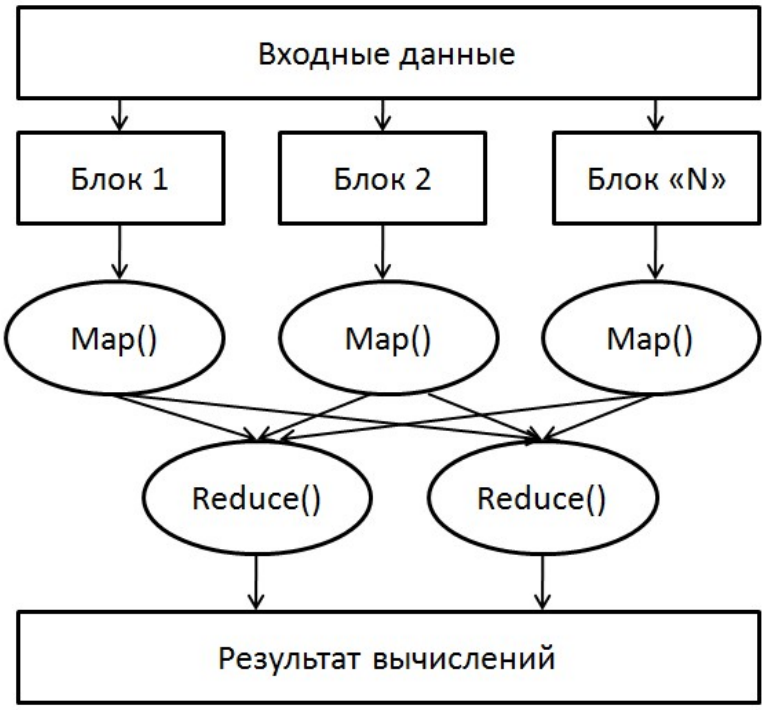


Рис. П2.2. Схема работы задачи MapReduce

На данной схеме перед этапом Map() происходит разделение данных на блоки (порции). На этапе Map() происходит обработка каждого блока на собственном вычислительном узле. Результатом этапа обработки является структура данных, которая содержит ключ и результат обработки. Следующим этапом является сортировка всех обработанных данных по ключу и их передача на блок Reduce(), после чего происходит объединение результатов, имеющих один ключ.

Для выполнения упражнения рекомендуется использовать IDE Eclipse.

Для использования библиотек Hadoop создайте проект, используя систему сборки Maven. Зависимость можно включить, добавив строчки в *pom.xml,* приведенные в Листинге П2.1.

Листинг П2.1. Maven зависимость для Apache Hadoop.

```
<dependency>
        <groupId>org.apache.hadoop</groupId>
        <artifactId>hadoop-core</artifactId>
        <version>0.20.2</version>
</dependency>
```

Так же можно загрузить пользовательские библиотеки вручную через Window->Preferences -> Java -> Build Path -> User Libraries.

Все **.jar* файлы, которые понадобятся в ходе выполнения работы, находятся в папке с установленным проектом в частности пути до библиотек:

$hadoop_folder/share/hadoop/mapreduce
$hadoop_folder/share/hadoop/common
$hadoop_folder/share/hadoop

Ход выполнения упражнения

1. Запустите задачу для обработки с помощью Hadoop. Дистрибутив содержит в себе примеры готовых MapReduce задач.[30]. Для запуска используется команда:

hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-2.6.0.jar <nameOfExample>.

Замените 2.6.0. на номер установленной у вас версии.

2. Напишите свой собственный пример MapReduce задачи. Наиболее простым вариантом является WordCount (подсчет слов), выполняющий подсчёт слов в тексте. Схема работы алгоритма WordCount в рамках MapReduce представлена на рис. П2.3.

Перед этапом Map() происходит разделение данных по строкам, затем производится подсчет слов для каждой строки в отдельности. Следующим шагом является сортировка подсчитанных слов по ключу. Завершающим этапом работы алгоритма является Reduce ().

Структурно проект должен содержать 3 класса: *WordCountMapper.java, WordCountReducer.java, WordCount.java.*

WordCountMapper.java содержащий функцию Map; Данный класс наследуется от класса *org.apache.hadoop.mapreduce.Mapper.*

Метод Map имеет входные параметры:

— key - входной ключ;

— value - входное значение;

— output - коллекция, которая хранит набор ключ/значение;

— reporter - утилита для отслеживания процесса.

[30] Список примеров MapReduce задач, доступных в Hadoop. [Электронный ресурс]. Режим доступа: // http://www.informit.com/articles/article.aspx?p=2190194&seqNum=3

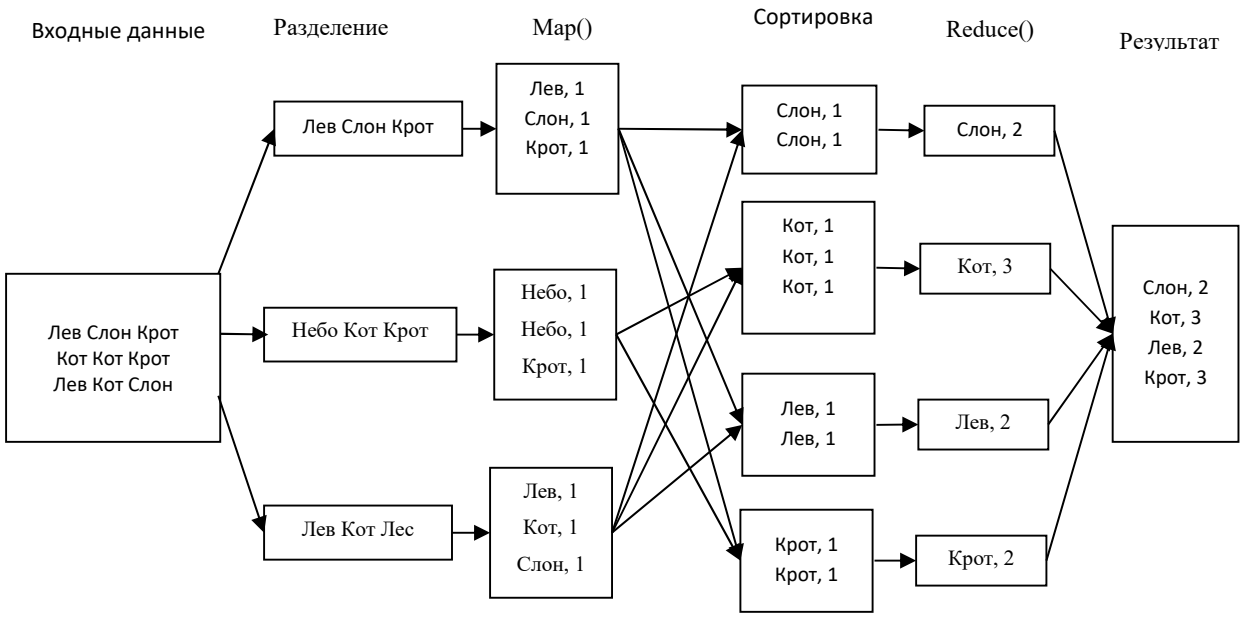


Рис. П2.3. Схема работы MapReduce на примере подсчета слов

WordCountReducer.java содержит реализацию функции Reduce. Данный класс наследуется от класса org.apache.hadoop.mapred.Reducer

WordCount.java является Main классом, который запускает задачу. Данный класс содержит метод, который утверждает задание на выполнение на кластере. Метод включает в себя:

— проверку входной и выходной спецификации для задания;

— подсчет входных разбиений для задания;

— установку необходимой информации о подсчете для распределенного кэша, при необходимости;

— копирование jar для задания, и конфигурация Map-Reduce системных директорий на распределенной файловой системе;

— передачу и утверждение задания на кластер и при необходимости отслеживание выполнения задания;

— обычно пользователь создает экземпляр класса org.apache.hadoop.mapreduce.Job(), с помощью которого описывает различные аспекты задания и затем использует метод объекта waitForCompletion(true) для выполнения задания и отслеживания состояния выполнения.

2.1. Сгенерируйте из проекта *.jar.

2.2. Создайте входной текстовый файл и поместите его в директорию input в HDFS.

2.3. Запустите вашу MapReduce задачу с входным текстовым файлом. Для запуска используется команда: *hadoop jar /path/to/your/jar.jat name.of.package.MainClass inputDirectoryInHDFS resultDirectoryInHDFS.*

Для успешной защиты упражнения требуется продемонстрировать следующие пункты, перечисленные ниже.

1. На своём компьютере запустить пример задачи MapReduce.

2. Показать свой проект, объяснить предназначение классов.

3. Запустить свою MapReduce задачу, показать результат выполнения.

УПРАЖЕННИЕ 3 – СОЗДАНИЕ БАЗЫ ДАННЫХ В HBASE

Цель данного упражнения — получение практических навыков администрирования и создания базы данных в HBase. Упражнение покрывает настройку и установку HBase и Hue, работу с HBase через командную строку (HBase shell) и работу с веб-интерфейсом Hue.

Теоретический материал

HBase — свободно распространяемая нереляционная распределённая база данных, входящая в экосистему Hadoop и разрабатываемая фондом Apache. Языком реализации является Java. Отличительной особенностью является использование файловой системы HDFS для хранения информации таблиц, т.е. представляет собой надстройку над HDFS. Относится к типу широких колоночных NoSQL баз данных, что позволяет обеспечивать отказоустойчивый способ хранения больших объёмов, разреженных данных.

Например, данные приведенные в таблице П2.2 в HBase будут выглядеть как в таблице П2.3.

Таблица П2.2 Формат хранения данных в стандартной таблице в БД.

ИД	Имя	Домашний телефон	Рабочий телефон	Рабочий адрес
1000	Иванов Иван	8-812-123-45-67	8-812-765-43-21	Невский пр. 1

Таблица П2.3. Формат хранения данных в HBase.

	Семейство столбцов: личные данные		Семейство столбцов: рабочие данные	
ИД	Имя	Телефон	Телефон	Адрес
1000	Иванов Иван	8-812-123-45-67	8-812-765-43-21	Невский пр. 1

При знакомстве с HBase нужно особое внимание уделить двум компонентам: MasterServer и RegionServer.

MasterServer является главным узлом в распределенной модели HBase, т.е. на кластере. Его главными ответственностями являются: распределение регионов по RegionServer и управление исполнением задач. Здесь под регионом понимается диапазон записей в таблице, которые относятся к определенному набору идущих друг за другом записей, отсортированных по ключу.

RegionServer — компонент, отвечающий за обработку и управление одним или несколькими регионами.

Каждый регион состоит из следующих компонент.

— PersistentStorage является основным хранилищем данных. Поскольку HBase все данных хранит на уровне файловой системы HDFS, то для этого потребовалось создать специальный формат файла HFile. В этом формате данные хранятся в порядке отсортированными по ключу записи.

— MemStore — это хранилище, выступающее в роли буфера для произведения операций записи. Поскольку при работе с HBase требуется поддержка операций обновления записей, а единичные операции при работе с распределенной файловой системой являются дорогостоящими в плане времени, то HBase предлагает использовать буферизированное хранилище. В этом хранилище осуществляется накопление данных до определенного значения. При достижении заданного значения данные целым набором выгружаются в HFile.

— BlockCache — аналог MemStore, используется в роли буфера на чтение. Использование данного буфера (кэш) позволяет осуществить экономию времени на чтении "горячих" данных (наиболее часто используемых).

— Write Ahead Log (WAL) — еще один компонент HBase, целю и ответственностью которого является восстановление данных при сбоях.

Такая необходимость появляется по причине использования буфера на запись (MemStore). Если данные находятся в буфере и физически еще не были записаны в HDFS, но произошел сбой, то WAL позволяет, используя специализированные логи, восстановить данные.

HBase содержит значительное число компонент и сервисов, которые нуждаются в координации между друг другом. Эту роль на себя берет Apache ZooKeeper[31]. ZooKeeper — это сервис, предназначенный для управления конфигурациями и синхронизацией сервисов.

Hue[32] — это пользовательский web-интерфейс, предназначенных для администрирования сервисов Hadoop (в том числе и HBase).

Архитектура HBase и взаимодействие с HDFS представлена на рис. П2.4.

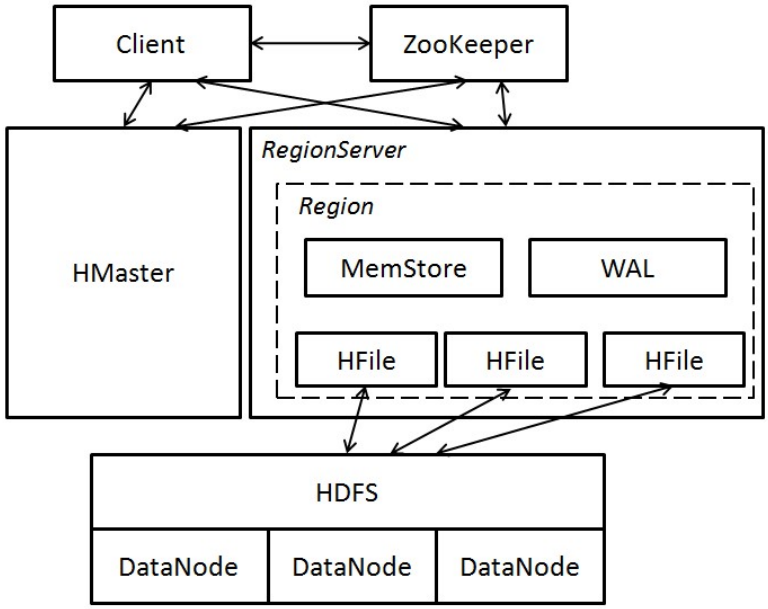


Рис. П2.4. Архитектура HBase[33]

Ход выполнения упражнения

1. Установите HBase в псевдораспределённом режиме. Обратите внимание на совместимость версий HBase с версиями Hadoop.

[31] Apache Zookeeper. Официальный сайт. [Электронный ресурс]. Режим доступа: // https://zookeeper.apache.org

[32] Hue. [Электронный ресурс]. Режим доступа: // http://gethue.com

[33] White T., Hadoop: The Definitive Guide. — 2-е изд. — O'Reilly, 2011. — 625 с.

2. Используя HBase shell, создайте таблицу и заполните её данными. Таблица должна содержать информацию о студентах:

— личные данные: ФИО, пол, дата рождения;

— рабочие данные: факультет, кафедра, номер группы;

— научная работа: ФИО научного руководителя, область научных исследований.

3. Установите Hue.

4. Настройте подключение к HDFS и HBase, изучите, с чем ещё может быть использован Hue.

5. Для интеграции с HBase необходимо запустить HBase Thrift Server, это можно сделать, используя команду *hbase thrift start*.

6. Создайте таблицу в HBase, используя Hue.

Для успешной защиты данного упражнения требуется продемонстрировать следующие пункты.

1. Показать создание таблицы в HBase, используя HBase shell.

2. Показать созданную таблицу, используя HBase shell.

3. Показать содержимое HDFS, используя Hue.

4. Создать таблицу в HBase используя Hue.

5. Показать содержимое HBase, используя Hue.

ПРИЛОЖЕНИЕ 3

УПРАЖНЕНИЯ ПО ТЕХНОЛОГИИ APACHE SPARK

ВВЕДЕНИЕ

Цель данного цикла упражнений – получение практических навыков администрирования и разработки приложений с использованием фреймворка Apache Spark.

Spark[34] — фреймворк, являющийся одним из проектов фонда Apache Software Foundation, который представляет собой набор open-source библиотек для разработки приложений распределённой обработки неструктурированных и слабоструктурированных данных. Apache Spark входит в экосистему проектов Hadoop. Apache Spark использует примитивы данных для обработки непосредственно в оперативной памяти. За счет такой in-memory обработки достигается значительный выигрыш во времени исполнения по сравнению с традиционной концепцией обработки данных Hadoop MapReduce для итеративных алгоритмов, которым требуется постоянное обращение к дисковому пространству.

Несмотря на то, что Apace Spark изначально разработан на языке Scala, он предоставляет широкий API для разработки приложений на различных языках программирования, отличных от родного: Java, Python, R

На сегодняшний день актуальной версий Apache Spark является версия 2.1.0.

В состав проекта Spark входят следующие компоненты, перечисленные ниже.

— Spark Core – реализация основных функциональных возможностей, к которым относятся: исполнение программ, планирование, обработка исключительных ситуаций и восстановление после ошибок, логирование, взаимодействие с источниками данных, управление и распределение памяти и др.

— Spark Streaming – компонент для обработки данных в потоковом режиме. Входные данные могут поступать с различных брокеров сообще-

[34] Sandy Ryza, Uri Laserson, Sean Owen, Josh Wills. Advanced Analytics with Spark, *O'Reilly Media*, 2015.

ний, таких как Apache Kafka, Flume или RabbitMQ, а также через предоставляемый им программные интерфейсы.

— MLlib – компонент, содержащий реализацию таких методов машинного обучения, как: классификация, регрессия, кластеризация, и др.. Отличительной особенностью этого компонента является то, что все методы способны работать в кластерной инфраструктуре и позволяют осуществлять гибкую горизонтальную масштабируемость.

— Spark SQL – компонент, позволяющих осуществлять работу со структурированными данными за счет разработки и исполнения SQL-запросов, в том числе и поддержке его диалекта Hive Query Language для Hadoop. В дополнении, компонент позволяет осуществлять перемешивание SQL-кода с кодом на таких языках как Java, Scala, Python, которые поддерживаются абстракциями RDD.

— GraphX – компонент-библиотека, поддерживающая параллельную обработку графов.

Приложения, созданные с помощью фреймворка Apache Spark могут быть запущены в одном из следующий режимов: полностью автономный режим (локальный), режим работы на кластере под управлением ресурс менеджеров Yarn или Mesos.

УПРАЖНЕНИЕ 1 – УСТАНОВКА SPARK И РАБОТА С SPARK SHELL

Цель данного упражнения – получение практических навыков установки и администрирования фреймворка Apache Spark. Упражнение покрывает установку и запуск Spark, работу с командными строками и с веб-интерфейсом фреймворка.

Теоретический материал

Приложения, созданные при помощи Apache Spark, запускаются как набор независимых процессов на кластере, управляемые объектом SparkContext[35] в основной программе, которая называется Program Driver. Для запуска на кластере SparkContext может подключаться к различным

[35] Документация по Spark Context. [Электронный ресурс]. Режим доступа: // https://spark.apache.org/docs/1.4.0/api/java/org/apache/spark/SparkContext.html

менеджерам ресурсов. При запуске а автономном режиме необходимо создавать SparkContext вручную. После подключения Spark может обращаться к исполнителям – процессам на узлах кластера, которые выполняют операции вычисления и хранения данных для основного приложения. Затем, он отправляет код приложения и задачи исполнителям. Обобщенная схема архитектуры Spark представлена на рис.П3.1

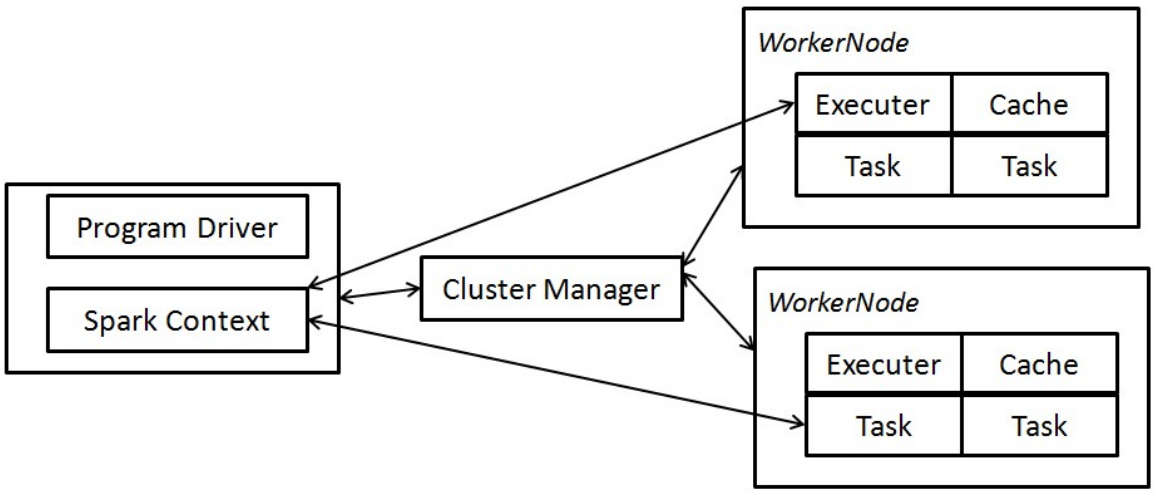


Рис. П3.1 Архитектура Spark

В представленной архитектуре стоит отметить несколько деталей:

— каждое приложение получает своего отдельного исполнителя, которое привязывается к нему на всё время исполнения, и выполняет задачи в несколько потоков. Это имеет определённые преимущества, но также означает, что данные не могут быть совместно использованы несколькими приложениями (одно приложение соответствует одной сущности SparkContext), без записи их на какое-либо внешнее хранилище;

— для Spark не имеет значения используемый менеджер ресурсов. Это может быть YARN, Mesos или любой другой менеджер;

— т.к. Program Driver планирует и запускает задачи на кластере, не следует использовать рабочие узлы «далеко» от основного, желательно иметь их в одной локальной сети.

Основная парадигма распределенных вычислений над большими данными заключается в том, что необходимо вычисления перенести к дан-

ным, а не данные к вычислениями, т.к. последнее крайне затратная операция.

Ход выполнения упражнения

1. Установите Apache Spark на локальный компьютер. Для работы со Spark рекомендуется, также как и в упражнениях по Hadoop (см. Приложение 2), создать отдельного пользователя системы. Например, назовите пользователя – sparkuser. Обратите внимание на то, что для установки Spark требуется установить sbt.

2. Используя команду *pyspark*, запустите командную строку Spark, используя язык Python, проделайте базовые операции, например, выведите на печать «Welcome to Spark».

3. Используя команду *spark-shell*, запустите командную строку Spark, используя язык Scala, проделайте базовые операции, как и в предыдущем пункте.

4. Откройте в браузере веб-интерфейс Spark, доступный по ссылке: http://localhost:4040 (адрес по-умолчанию), и изучите его возможности. Веб-интерфейс доступен, когда запущена командная строка.

Для успешной сдачи упражнения требуется:

— запустить Spark на локальном компьютере;

— запустить командную строку (любую) и вывести в ней на печать «Welcome to Spark»;

— запустить веб-интерфейс Spark.

УПРАЖНЕНИЕ 2 – СОЗДАНИЕ И ОПЕРАЦИИ НАД RDD

Цель данного упражнения – получение навыков работы с основными типами данных Spark - RDD[36]. Упражнение покрывает работу с объектом SparkContext, загрузку данных в Spark, их обработку и выгрузку. В упражнении подразумевается написание своего приложения, использующего RDD для обработки данных.

Теоретический материал

Основным понятием и базовой абстракцией в Spark является RDD (Resilient Distributed Dataset, устойчивый распределенный набор данных) – это простая, неизменяемая, распределенная, отказоустойчивая коллекция объектов. Такие коллекции могут обрабатываться параллельно[37]. Данные любых типов могут быть помещены в RDD.

Создавать RDD можно несколькими способами: либо через загрузку внешнего набора данных, либо после распределения коллекции из основной программы. Над RDD можно осуществлять операции двух типов: преобразования (transformation) и действия (actions).

Преобразования – это операции над RDD, результатом которых является создание нового набора RDD. Примерами преобразования могут служить: отображение, фильтрация, объединение и т.д..

Действия – это операции над RDD, которые возвращают конечное значение, получаемое в результате некоторых вычислений в RDD. Примерами таких операция служат: редукция, подсчет и т.д..

Преобразования осуществляются в «ленивом» (lazy) режиме. Результат не вычисляется сразу, он помещается в стек и запоминается в нем до тех пор, пока не встретится действие. Именно результат действия возвращается основной программе и он содержит в том числе и все результаты преобразований. Такой подход значительно повышает производительность Apache Spark.

---

[36] Документация Spark по RDD. [Электронный ресурс]. Режим доступа: // http://spark.apache.org/docs/1.2.1/programming-guide.html#resilient-distributed-datasets-rdds

[37] Карау Х., Конвински Э. Изучаем Spark. Молниеносный анализ данных. — ДМК Пресс, 2015. — 304 с.

Для выполнения упражнения рекомендуется использовать IDE Eclipse. Для создания проекта может быть использован любой язык, поддерживаемый Spark. В данных указаниях примеры будут приведены для языка Java.

Для использования библиотек Spark создайте проект, используя систему сборки Maven. Зависимость можно включить, добавив строчки в *pom.xml,* приведенные в Листинге 1. Версия должна быть выбрана в соответствии в установленной на компьютере.

Листинг П3.1. Maven зависимость для Apache Spark.

```
<dependency>
   <groupId>org.apache.spark</groupId>
   <artifactId>spark-core_2.10</artifactId>
   <version>1.6.3</version>
</dependency>
```

Также можно загрузить пользовательские библиотеки вручную через Window->Preferences -> Java -> Build Path -> User Libraries.

Все **.jar* файлы, которые понадобятся в ходе выполнения упражнения, доступны на официальном сайте Spark.

Следует отметить, что каждому отдельному приложению Spark соответствует один объект SparkContext, поэтому он должен быть проинициализирован один раз. Пример инициализации представлен в Листинге П3.2.

Листинг П3.2. Создание и инициализация объекта SparkContext.

```
SparkConf conf = new SparkConf();
conf.setAppName("my spark application");
conf.setMaster("local[*]");
JavaSparkContext sc = new JavaSparkContext(conf);
```

Перед созданием объекта SparkContext, необходимо создать объект конфигурации, который будет передан в конструктор контекста. У объекта конфигурации необходимо задать имя приложения, используя метод *setAppName()*, имя может быть любое. Задание имени приложения является обязательным условием для запуска приложения. Метод *setMaster()* задает менеджера ресурсов, в данном примере Spark запускается локально, но приветствуется запуск на стороннем менеджере, например YARN, т.к.

ожидается, что в после выполнения упражнений из Приложения 2 данный менеджер уже присутствует в системе, как часть фреймворка Hadoop.

Ход выполнения упражнения

1. Ознакомьтесь с API RDD: трансформациями и действиями.

2. Создайте проект, используя Spark, выполняющий следующие задачи:

— открывает и считывает файл с набором слов (содержание файла может быть любым: набор связанных слов, текст, слова песни и т.д.);

— создаёт RDD, используя данные из файла;

— осуществляет его обработку с использованием трансформаций и действий;

— возвращает список 15 наиболее популярных слов, отсортированных по количеству возникновения в тексте в убывающем порядке;

— записывает данный список в файл.

Используя язык, Java обратите внимание на то, что допускается использование конструкций языка Scala, особенно полезной может быть конструкция *Tuple2*[38] (эквивалент класса *Pair*, используемого в Java) в связи с тем, что она имеет ряд методов, упрощающих обработку данных.

Для успешной защиты упражнения требуется продемонстрировать следующие пункты:

— показать свой проект, объяснить структуру кода и предназначение классов;

— запустить приложение, показать результат выполнения в виде текстового файла.

[38] Документация по классу Tuple. [Электронный ресурс]. Режим доступа: // http://www.scala-lang.org/api/2.9.3/scala/Tuple2.html

УПРАЖНЕНИЕ 3 – СОЗДАНИЕ DATAFRAME

Цель данного упражнения – получение практических навыков работы со структурой DataFrame[39]. В упражнении подразумевается создание приложения, которое будет загружать данные в Spark, создавать DataFrame, обрабатывать их, выгружать результаты обработки и записывать их в выходной файл.

Теоретический материал

Начиная с версии Spark 1.3 была добавлена структура DataFrame. Она является распределённой коллекцией данных, на подобии RDD, но организованна в виде именованных колонок. Идеологически DataFrame схожи с таблицами в реляционных базах данных или с DataFrame в R[40], а также Python Pandas[41]. Но отличительной особенностью Spark DataFrame является оптимизация для распределённых вычислений.

DataFrame поддерживает возможность инициализации и создания экземпляров на основе структурированных или полуструктурированных файлов, таких как JSON, XML или Parquet[42]. Присутствует поддержка создания на основе данных из РСУБД посредством JDBC/ODBC, а также другими способами через внешние программы-коннекторы сторонних разработчиков.

API для работы с DataFrame доступны из всех поддерживаемых Spark языков. Также можно осуществлять обращение к таблицам, через SQL-подобные языки, например, Hive.

---

[39] Документация Spark по DataFrame. [Электронный ресурс]. Режим доступа: // https://spark.apache.org/docs/1.6.0/sql-programming-guide.html#dataframes

[40] R language fundamentals. Data frames. Steven Buechler. [Электронный ресурс]. Режим доступа: //
https://www3.nd.edu/~steve/Rcourse/Lecture2v1.pdf

[41] Python Pandas. Официальный сайт. [Электронный ресурс]. Режим доступа: // http://pandas.pydata.org/

[42] Apache Parquet. Официальный сайт. [Электронный ресурс]. Режим доступа: // https://parquet.apache.org

При работе с DataFrame требуется объект SQLContext[43]. Для создания объекта SQLContext требуется уже созданный объект SparkContext (передаётся в конструктор SQLContext). Для его использования необходимо добавить дополнительную зависимость в Maven, представленную в листинге П3.3.

Листинг П3.3. Maven зависимость для Spark SQL.

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.10</artifactId>
  <version>1.6.3</version>
</dependency>
```

DataFrame предоставляет широкий набор методов и функций, например: agg, columns, count, distinct, drop, dropDuplicates, filter, groupBy, orderBy, registerTable, schema, show, select, where, withColumn и др.

Ход выполнения упражнения

1. Ознакомьтесь с API DataFrame.[44]

2. В листинге П3.4 представлен пример входных данных в формате JSON. Данные имеют поля: имя, возраст и список технологий, которыми владеет человек.

Листинг П3.4. Пример входных данных.

```
{"name": "Evgeny Ivanov", "age": 37, "keywords": ["spark", "java", "spring"]}
{"name": "Ksenia Petrova", "age": 30, "keywords": ["sql", "hive", "hadoop", "impala"]}
{"name": "Michail Smirnov", "age": 42, "keywords": ["spring boot", "java", "spring"]}
{"name": "Ivan Golubev", "age": 44, "keywords": ["java", "maven", "gc", "hadoop"]}
{"name": "Alexandr Victorov", "age": 38, "keywords": ["docker", "spring", "groovy"]}
```

[43] Документация по SQLContext. Официальный сайт. [Электронный ресурс]. Режим доступа: // https://spark.apache.org/docs/1.6.1/sql-programming-guide.html#starting-point-sqlcontext

[44] Документация по набору операций для DataFrame. [Электронный ресурс]. Режим доступа: // https://spark.apache.org/docs/1.6.0/sql-programming-guide.html#dataframe-operations

Для упражнения можно использовать, как приведённый пример, так и реальные данные, полученный социальных сетей.

3. Основываясь на данных из социальной сети LinkedIn (или HeadHunter), требуется определить самую популярную технологию, а затем найти разработчиков, которые будут согласны работать с учётом зарплаты 15 000 рублей в месяц, при условии наличия у них опыта работы с этой технологией.

Обратите внимание на то, что в примере данных поля зарплаты нет. При отсутствии данных о зарплате в социальной сети, предлагается использовать формулу [зарплата] = [возраст] * 10 * [количество технологий, которые знает человек].

На выходе требуется вывести список имён людей, соответствующих требованиям.

Для успешной защиты упражнения требуется продемонстрировать следующие пункты:

— показать проект, объяснить структуру кода и предназначение классов;

— запустить приложение, показать результат выполнения в виде текстового файла.

ПРИЛОЖЕНИЕ 4

ПРИМЕРЫ ИТОГОВОГО ТЕСТОВОГО ЗАДАНИЯ

ИТОГОВЫЙ ТЕСТ – ВАРИАНТ 1

Итоговый тест по дисциплине
«Наука о данных и аналитика больших
объемов информации»

_____/_____
(Ф.И.О) (группа)

_____
(дата)

_____
(подпись)

Номер	Ответ
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

Номер	Ответ
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	
26	
27	
28	
29	
30	

1. Дайте определение термину «большие данные» (англ., «big data»).
    A. «Большие данные» - это комплексный набор инструментов обработки структурированных данных колоссальных объемов.
    B. «Большие данные» - это подход хранения неструктурированных данных колоссальных объемов.
    C. «Большие данные» - это комплексный набор методов и технологий обработки и передачи структурированных, полуструктурированных и неструктурированных наборов данных колоссальных объемов за разумное время, а также хранения данных
    D. «Большие данные» - это подключение к глобальной сети большого числа окружающих нас объектов.

2. Выберите главные характеристики «больших данных».
    A. Огромный объем данных, сложность типов данных и их структуры, скорость появления новых данных.
    B. Огромный объем данных, скорость обработки больших данных, скорость появления новых данных.
    C. Огромный объем данных, достоверность данных, ценность накопленной информации.
    D. Огромный объем данных, скорость анализа данных, скорость обработки данных.

3. Какой тип данных имеет наибольший объем на сегодняшний день?
    A. Структурированные
    B. Полуструктурированные
    C. Квази-структурированные
    D. Неструктурированные.

4. Какой из нижеперечисленных принципов работы не применятся к подходам, используемым при работе с «большими данными»?
    A. Горизонтальная масштабируемость.
    B. Вертикальная масштабируемость.
    C. Отказоустойчивость и отсутствие “single point of failure” в системе.
    D. Распределенное хранение.

5. Какая из характеристик не является основной для «Больших данных»
    A. Скорость (Velocity).
    B. Объём (Volume).
    C. Многообразие (Variety).
    D. Качество (Quality).

6. Данные текстовых файлов с определенными паттернами для их обработки, например, XML, являются...
    A. Структурированными.
    B. Полуструктурированными.
    C. Квазиструктурированными.
    D. Неструктурированными.

7. Что означает термин «бизнес-аналитика» («Business Intelligence») в информационных технологиях?
    A. «Бизнес-аналитика» - это комплексный набор методов для создания бизнес-планов.
    B. «Бизнес-аналитика» - это методы и инструменты для перевода необработанной информации в осмысленную, удобную для восприятия форму.
    C. «Бизнес-аналитика» - это файлы, содержащие информацию о бизнес-плане.
    D. «Бизнес-аналитика» - это технологии, направленные на развитие бизнеса.

8. Какую возможность не предоставляют современные платформы для «бизнес-аналитики» («Business Intelligence» платформы)?
    A. Возможность подключения к различным источникам данных (от файла Excel до универсального ODBC подключения).
    B. Возможность построения отчетов в различных форматах (Excel, HTML, PDF и т. д.).
    C. Возможность интерактивной работы с данными (формирование отчетов «на лету»).
    D. Возможность формирования прогнозов.

9. Что не включает в себя ETL- процесс?
    A. Извлечение данных.
    B. Трансформацию и очистку данных.
    C. Загрузку данных в хранилище.
    D. Поиск данных.

10. Выберите правильное определение понятия «Хранилище данных»:
    A. Предметно-ориентированная информационная база данных, специально разработанная и предназначенная для подготовки отчётов и бизнес-анализа.
    B. Программная система для доступа к структурированной, полуструктурированной и неструктурированной информации.
    C. Комплексная распределенная система из информационных хранилищ, предназначенная для обработки кластером.
    D. Все выше перечисленное.

11. Дайте определение понятию «наука о данных» («data science»)?

A. Это методы и инструменты для перевода необработанной информации в осмысленную, удобную форму.

B. Компьютерная техника извлечения знаний, которая использует ИИ для распознавания образов и выделения значимых закономерностей из данных, находящихся в хранилищах или входных, или выходных потоках.

C. Инструменты для анализа данных, построения отчетов и запросов могут помочь бизнес-пользователям преодолеть море данных для того, чтобы синтезировать из них значимую информацию.

D. Раздел информатики, изучающий проблемы анализа, обработки и представления данных в цифровой форме.

12. Дайте определение процессу "интеллектуальный анализ данных"?

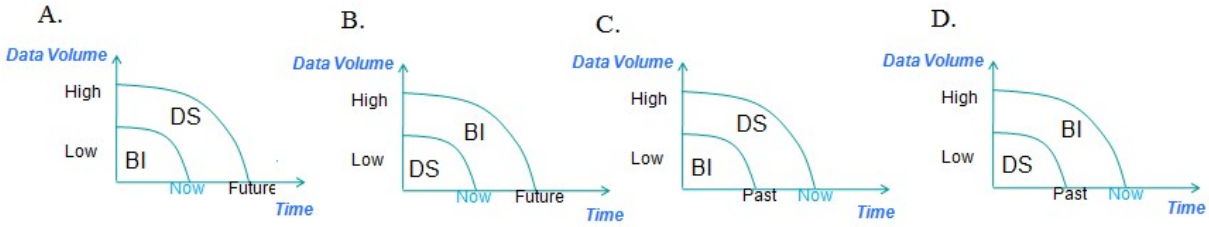
A. Методы и инструменты для перевода необработанной информации в осмысленную, удобную форму.

B. Процесс, ориентированный на бизнес-пользователя и включающий доступ и исследование информации, ее анализ, выработку интуиции и понимания, которые ведут к улучшенному и неформальному принятию решений.

C. Процесс обнаружения в сырых данных ранее неизвестных, нетривиальных, практически полезных, доступных интерпретации знаний, необходимых для принятия решений в различных сферах человеческой деятельности.

D. Компьютерная техника извлечения знаний, которая использует искусственный интеллект для распознавания образов и выделения значимых закономерностей из данных, находящихся в хранилищах или входных, или выходных потоках.

13. Выберите правильный график отношения терминов «бизнесс-аналитика» ("Business Intelligence") и «наука о данных» ("Data Science").



14. AMQP (Advanced Message Queuing Protocol) - это...
    A. Протокол, который задает набор правил для передачи почты.
    B. Набор правил и действий, позволяющий осуществлять соединение и обмен данными между устройствами.
    C. Открытый протокол для передачи сообщений между компонентами системы.
    D. Стандартный протокол почтового соединения.

15. В чем состоят отличия Apache Kafka от традиционных систем передачи сообщений?
    A. Изначально спроектирован как распределенная система.
    B. Поддерживает объединение пользователей в группы.
    C. Обеспечивает возможность хранения данных.
    D. Всё из вышеперечисленного.

16. Разбиение системы на более мелкие структурные компоненты и разнесение их по отдельным физическим серверам (или их группам), и (или) увеличение количества серверов, параллельно выполняющих одну и ту же функцию это…
    A. Горизонтальное масштабирование.
    B. Вертикальное масштабирование.
    C. Master-slave репликация.
    D. Peer-to-peer репликация.

17.Выберите неверное утверждение:

A. Подход «интеллектуального анализа данных» («Data Mining») использует методы машинного обучения.
B. «Интеллектуальный анализ данных» («Data Mining») направлен, в том числе и на поиск и извлечение скрытых зависимостей.
C. Методы используемые для «интеллектуального анализа данных» («Data Mining») могут быть применены исключительно для работы с «большими данными».
D. Деревья решений являются одним из паттернов представления знаний, добываемых «интеллектуального анализа данных» («Data Mining»).

18.Эволюционное программирование связано с …

A. Изучением и сбором информации на продолжительном промежутке времени с целью прогноза будущих состояний системы
B. Поиском зависимости целевых переменных от остальных в форме функций какого-то определенного вида
C. Системами машинного обучения
D. Робототехникой

19.Ошибкой второго рода в статистике является

A. Ошибка не обнаружить различия или связи, которые на самом деле существуют.
B. Ошибка обнаружить различия или связи, которые на самом деле не существуют.
C. Ошибка обнаружить различия или связи, которые на самом деле существуют.
D. Ошибка не обнаружить различия или связи, которые на самом деле не существуют.

20. Алгоритм на каждой итерации вычисляет центр масс для каждого кластера, полученного на предыдущем шаге, затем векторы разбиваются на кластеры вновь в соответствии с тем, какой из новых центров оказался ближе по выбранной метрике.
    A. Алгоритм k-means
    B. Алгоритм c-means
    C. Иерархический алгоритм
    D. Алгоритм послойной кластеризации

21. Что является отличительной особенностью NoSQL?
    A. Вертикальная масштабируемость
    B. Сложный API
    C. Нереляционные базы данных
    D. Жестко заданная схема данных

22. Какое свойство РСУБД означает, что транзакции не нарушают согласованность данных, то есть они переводят базу данных из одного корректного состояния в другое.
    A. Atomicity
    B. Consistency
    C. Isolation
    D. Durability

23. Распределите по уровню «сложности» хранимых структур данных следующие классы NoSQL баз данных (по возрастанию).
    A. Документо-ориентированные хранилища;
    B. Графо-ориентированные хранилища;
    C. Хранилище колоночного типа;
    D. Хранилище для Ключ/Значение.

24.Что такое «Apache Hadoop»?

A. Методы и инструменты для перевода необработанной информации в осмысленную, удобную форму
B. Платформа для построения приложений, способных обрабатывать огромные объемы данных
C. Распределенный координационный сервис
D. Технология распределенных вычислений

25.Что является достоинствами парадигмы «Map Reduce»?

A. Оптимальная производительность;
B. Эффективное применение в маленьких кластерах с небольшим объемом данных;
C. Масштабируемость;
D. Отказоустойчивость, что позволяет вычислять задачи, даже если некоторые вызовы завершились аварийно.

26.Что из нижеперечисленного не является одним из способов работы «Apache Hadoop»?

A. Автономный режим.
B. Псевдо-распределенный режим.
C. Полностью распределенный режим.
D. Связанный режим.

27.Выберите неверное утверждение из нижеперечисленных.

A. Концепция RDD – неизменяемая отказоустойчивая распределенная коллекция объектов, которые можно обрабатывать параллельно;
B. Трансформации – это операции (например, отображение, фильтрация, объединение и т.д.), совершаемые над RDD;
C. Лямбда-архитектура представляет собой индивидуальный подход, направленный на применение определенных функции к определенному набору данных;
D. «Spark Streaming» поддерживает обработку потоковых данных в реальном времени.

28. Структура данных в таких базах представляет собой набор узлов, связанных между собой ссылками. При этом и узлы и ссылки могут обладать некоторым количеством атрибутов.
    A. Ширококолоночные БД.
    B. Документоориентированные БД.
    C. Ключ/Значение БД.
    D. Графоориентированные БД.

29. Графический способ подачи информации, данных и знаний, целью которого является быстро и чётко преподносить сложную информацию, представляет собой длинную «простыню» с картинками и текстом.
    A. Презентация.
    B. Инфографика.
    C. Бизнес аналитика.
    D. Картограмма.

30. Что такое дедупликация данных?
    A. Технология, при помощи которой обнаруживаются и исключаются избыточные данные в дисковом хранилище
    B. Платформа для построения приложений, способных обрабатывать огромные объемы данных
    C. Процесс, ориентированный на бизнес-пользователя и включающий доступ и исследование информации, ее анализ, выработку интуиции и понимания, которые ведут к улучшенному и неформальному принятию решений
    D. Процесс создания (или выбора) модели для предсказания вероятности наступления некоторого события

Таблица П4.1

Ответы на тесты

Номер	Ответ
1	C
2	A
3	D
4	B
5	D
6	B
7	B
8	D
9	D
10	A
11	D
12	C
13	C
14	C
15	D

Номер	Ответ
16	A
17	C
18	B
19	A
20	A
21	C
22	B
23	D C A B
24	B
25	C D
26	D
27	C
28	D
29	B
30	A

ИТОГОВЫЙ ТЕСТ – ВАРИАНТ 2

Итоговый тест по дисциплине
«Наука о данных и аналитика больших
объемов информации»

_____/_____
(Ф.И.О) (группа)

_____
(дата)

_____
(подпись)

Номер	Ответ
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

Номер	Ответ
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	
26	
27	
28	
29	
30	

1. Что означает термин «большие данные» (англ., «Big Data») в информационных технологиях?
    A. «Большие данные» - это комплексный набор методов для создания файлов большого объёма.
    B. «Большие данные» - это комплексный набор методов обработки структурированных и неструктурированных данных колоссальных объемов за разумное время.
    C. «Большие данные» - это файлы с большим количеством данных.
    D. «Большие данные» - это представление времени, дня, месяца и года в качестве значения количества миллисекунд, прошедших с начала нашей эры.

2. Данные имеющие определенный тип, формат и структуру являются…
    A. Структурированными.
    B. Полуструктурированными.
    C. Квазиструктурированными.
    D. Неструктурированными.

3. Какими из перечисленных пунктов характеризуются «Большие данные»?
    A. Большой объем, высокая скорость поступления и выбытия данных и большое их разнообразие.
    B. комплексный набор методов обработки структурированных и неструктурированных данных колоссальных объемов.
    C. Данные больших размеров, высокой изменчивости, и большого разнообразия.
    D. Все вышеперечисленное.

4. Что такое Apache Hadoop?
    A. Apache Hadoop – это методы и инструменты для перевода необработанной информации в осмысленную, удобную для восприятия форму.
    B. Apache Hadoop – это платформа (фреймворк) для построения приложений, способных обрабатывать огромные объемы данных.
    C. Apache Hadoop – это распределенный координационный сервис по обработке данных.
    D. Apache Hadoop – это технология распределенных вычислений.

5. Дайте определение алгоритму Map Reduce.
    A. Алгоритм Map Reduce – это модель распределенных вычислений, предназначенная для параллельных вычислений над очень большими (до нескольких петабайт) объемами данных.
    B. Алгоритм Map Reduce – это набор компонентов и интерфейсов для распределенных файловых систем и общего ввода-вывода.
    C. Алгоритм Map Reduce – это распределенная файловая система, работающая на больших кластерах типовых машин.
    D. Алгоритм Map Reduce – это распределенный сервис для коллекционирования, сбора, и перемещения больших массивов данных.

6. Перечислите несколько достоинств алгоритма Map Reduce.
    A. Оптимальная производительность.
    B. Эффективное применение в маленьких кластерах с небольшим объемом данных.
    C. Масштабируемость.
    D. Отказоустойчивость, что позволяет вычислять задачи, даже если некоторые вызовы завершились аварийно.

7. Какой режим используется для того, чтобы на локальной машине использовать фреймворк Apache Hadoop как кластер, состоящий из одного узла?
    A. Автономный режим (Standalone Mode).
    B. Псевдораспределенный режим (Pseudo Distributed Mode).
    C. Полностью распределенный режим (Fully Distributed Mode).
    D. Такого режима не существует.
8. Как называется распределенное хранилище данных, входящее в экосистему Apache Hadoop, отвечающее за управление данными, хранящимися в HDFS, и предоставляющее язык запросов на базе SQL для работы с этими данными?
    A. Hive.
    B. Hbase.
    C. Sqoop.
    D. Pig.
9. Что такое Pig в экосистеме Hadoop?
    A. Pig – это нереляционная распределенная база данных, исполняемая поверх HDFS.
    B. Pig – это сервис для записи и планировки заданий Hadoop.
    C. Pig – это распределенный сервис для коллекционирования, сбора, и перемещения больших массивов данных.
    D. Pig – это язык управления потоком данных и исполнительная среда для анализа больших объемов данных.
10. Что такое Apache Spark?
    A. Apache Spark – это библиотека для манипуляций над графами и выполнения с ними параллельных операций.
    B. Apache Spark -это базовый механизм для крупномасштабной параллельной и распределенной обработки данных.
    C. Apache Spark - это проект Apache, который позиционируется как инструмент для «молниеносных кластерных вычислений».
    D. Apache Spark -это нереляционная распределенная база данных, исполняемая на верху HDFS.

11. Выберите неверное утверждение.
    A. NoSQL базы данных эффективны в хранении плохо структурированной информации.
    B. NoSQL базы данных разработаны для хранения структурированной информации.
    C. NoSQL базы данных эффективны для аналитической обработки данных.
    D. NoSQL базы данных не поддерживают JOIN операции в полном объеме SQL-спецификации.

12. Увеличение производительности каждого компонента системы с целью повышения общей производительности называется …?
    A. Горизонтальное масштабирование.
    B. Вертикальное масштабированиие.
    C. Master- slave репликация.
    D. Peer-to-peer репликация.

13. Какое свойство означает, что транзакции не нарушают согласованность данных, то есть они переводят базу данных из одного корректного состояния в другое?
    A. Atomicity.
    B. Consistency.
    C. Isolation.
    D. Durability.

14. Перечислите плюсы наличия репликации серверов в системе?
    A. Понижение отказоустойчивости системы в целом.
    B. Понижение нагрузки отдельных серверов системы.
    C. Создание дубликата БД.
    D. Записи всегда делаются на Master сервер.

15.Дайте определение репликации?

A. Это последовательность действий, которую нужно выполнить на наборе входных данных для эффективного достижения цели аналитики с помощью выбранных методов анализа.
B. Разделение баз данных на отдельные части так, чтобы каждую из них можно было вынести на отдельный сервер.
C. Методы и инструменты для перевода необработанной информации в осмысленную, удобную форму.
D. Копирование данных на другие узлы при обновлении.

16.Дайте определение «науке о данных» (англ., «Data Science»)?

A. Наука о данных - это методы и инструменты для перевода необработанной информации в осмысленную, удобную форму.
B. Наука о данных - это компьютерная техника извлечения знаний, которая использует ИИ для распознавания образов и выделения значимых закономерностей из данных, находящихся в хранилищах или входных, или выходных потоках.
C. Наука о данных - это инструменты для анализа данных, построения отчетов и запросов могут помочь бизнес-пользователям преодолеть море данных для того, чтобы синтезировать из них значимую информацию.
D. Наука о данных - это раздел информатики, изучающий проблемы анализа, обработки и представления данных в цифровой форме.

17.Основное умение специалистов по изучению данных (аналитиков данных)?

A. Умение находить наиболее важные элементы в хранимой информации.
B. Уметь прогнозировать исход работы системы.
C. Находить скрытые логические связи в системе собранной информации.
D. Отличать неструктурированные данные от структурированных.

18. Какое из перечисленных качеств не подходит человеку, занимаемому должность «аналитика»?
    A. Оперативность.
    B. Бизнес – ориентированность.
    C. Замкнутость.
    D. Креативность.

19. Выберите неверное утверждение.
    A. RabbitMQ – это брокер сообщений, основная цель которого принимать и отдавать сообщения.
    B. RabbitMQ позволяет взаимодействовать различным программам при помощи протокола AMQP.
    C. RabbitMQ позволяет осуществлять распределенную обработку данных.
    D. RabbitMQ является отличным решением для построения SOA (сервис-ориентированной архитектуры).

20. Что из этого не является режимом работы очереди RabbitMQ?
    A. Рассылка типа точка-точка (direct).
    B. Рассылка сообщений по шаблону (topic).
    C. Узко вещательная рассылка сообщений (narrow-out).
    D. Широко вещательная рассылка сообщений (fan-out).

21. Дайте определение для ETL.
    A. ETL - это процесс извлечения данных из хранилища, их трансформация, обработка и загрузка в целевое хранилище.
    B. ETL - это один из основных процессов предподготовки данных.
    C. ETL - это один из основных сервисов, работающих с хранилищами данных.
    D. ETL - это один из основных процессов в управлении хранилищами данных.

22. Как называется инструмент, построенный вокруг экосистемы Apache Hadoop, для отправки сообщений в HDFS и HBase?
    A. Apache Kafka.
    B. RabbitMQ.
    C. Talend Open Studio.
    D. Apache Flume.

23. Что не является целью процесса «бизнес аналитики» (англ., «Business Intelligence»)?
    A. Интерпретация большого количества данных.
    B. Моделирование исходов различных вариантов действий.
    C. Модификация существующего программного обеспечения.
    D. Отслеживание результатов решений.

24. Какой из этапов не включает в себя ETL- процесс?
    A. Извлечение данных.
    B. Трансформацию и очистку данных.
    C. Загрузку данных в хранилище.
    D. Поиск данных.

25. Дайте определение термину «хранилище данных»?
    A. Хранилище данных - это предметно-ориентированная информационная база данных, специально разработанная и предназначенная для подготовки отчётов и бизнес-анализа.
    B. Хранилище данных - это носитель информации или удаленный сервер, способные хранить, получать, отдавать и обрабатывать данные.
    C. Хранилище данных - это система из информационных хранилищ, предназначенных для хранения информации обрабатываемой кластером.
    D. Все вышеперечисленные определения верны.

26. Какие цели ставит перед собой «наука о данных» (англ., «Data Science»)?
    A. Ответить на вопрос что случилось ранее и как можно оптимизировать бизнес-процесс.
    B. Сделать прогноз на будущее на основе информации из прошлого.
    C. Проанализировать исторические данные, полученные на текущий момент и предоставить отчеты.
    D. Анализ не только структурированных данных, но и неструктурированных данных для поиска скрытых зависимостей.

27. Дайте определение термину «Предиктивное моделирование»?
    A. Предиктивное моделирование – это процесс создания (или выбора) модели для предсказания вероятности наступления некоторого события.
    B. Предиктивное моделирование – это методы и инструменты для перевода необработанной информации в осмысленную, удобную форму.
    C. Предиктивное моделирование – это последовательность действий, которую нужно выполнить на наборе входных данных для эффективного достижения цели аналитики с помощью выбранных методов анализа.
    D. Предиктивное моделирование – это компьютерная техника извлечения знаний, которая использует искусственный интеллект для распознавания образов и выделения значимых закономерностей из данных, находящихся в хранилищах или входных, или выходных потоках.

28. Какая из перечисленных задач относится к задам машинного обучения без учителя?

A. Задача классификации.
B. Задача регрессии.
C. Задача кластеризации.
D. Нет верного ответа.

29. Чем отличаются ошибки первого и второго рода при принятии решений?

A. Ошибка первого рода значительнее, нежели второго.
B. Ошибка второго рода не обнаруживает различия, которые есть, а первого обнаруживает, которых нет.
C. Ошибка второго рода значительнее, нежели первого.
D. Ошибка первого рода не обнаруживает различия, которых нет, а второго обнаруживает.

30. Что такое линейные классификаторы?

A. Линейные классификаторы – это идея о логических закономерностях происхождения данных.
B. Линейные классификаторы – это идея об измеримости пространства.
C. Линейные классификаторы – это идея о наличии линейных разделяющих плоскостей между классами в пространстве объектов.
D. Правильный ответ отсутствует.

Таблица П4.2

Ответы на тесты

Номер	Ответ
1	B
2	A
3	D
4	B
5	A
6	C, D
7	A
8	A
9	D
10	C
11	B
12	B
13	B
14	B, C
15	D

Номер	Ответ
16	D
17	C
18	C
19	C
20	C
21	A
22	D
23	C
24	D
25	A
26	B, D
27	A
28	C
29	B
30	C

ПРИЛОЖЕНИЕ 5

ПРИМЕРЫ ЭКЗАМЕНАЦИОННЫХ ВОПРОСОВ

Список экзаменационных вопросов
по дисциплине
«Наука о данных и аналитика больших объемов информации»

1. Локальные вычисления и связанные с ними проблемы производительности.
2. Способы повышения производительности вычислений в зависимости от класса решаемой задачи.
3. Определение термина «большие данные». Характеристики, свойства и классы информации.
4. Классификация способов обработки информации. Программная поддержка каждого из классов.
5. Экосистема «Apache Hadoop». Основные компоненты и их назначение.
6. Классификация типов задач анализа данных.
7. Вычислительная парадигма «MapReduce» и особенности реализации в фреймворках «Apache Hadoop» и «Apache Spark».
8. Lambda/Kappa-архитектуры программной системы обработки информации
9. Программная система «Apache Spark». Основные компоненты и их назначение. Отличие от «Apache Hadoop».
10. Теорема CAP. Классы баз данных на основе теоремы CAP.
11. «NoSQL» базы данных и их отличие от РСУБД. Классы «NoSQL» баз данных.
12. Брокеры сообщений. Назначение и основные свойства.
13. Брокер сообщений – «RabbitMQ». Назначение и основные свойства.
14. Брокер сообщений «Apache Kafka». Назначение и основные свойства.

15. Инфраструктура кластерного центра. Основные способы взаимодействия с кластером.
16. «In-memory» базы данных, их назначение и характеристики. Основные инструментальные средства.
17. Индексация информации в базе данных и поиск по ней. Инструменты «Elasticsearch» и «Kibana» (ELK).
18. Информационный поиск. Программные средства поддержки поиска текстовой информации: «Apache Lucene» и «Apache Solr».
19. «Кластер», «Облако», «Data Grid». Основные отличия и характеристики.
20. Распределенные файловые системы на примере HDFS. Репликация данных.
21. Денормализация баз данных для проведения анализа. Отличие от нормализации и необходимость применения.
22. Жизненный цикл проекта по аналитике данных.
23. Отличие «науки о данных (data science)» от традиционной «бизнес-аналитики (business intelligence)».
24. Роли участников команды, необходимые для успешного проекта по аналитике данных.
25. Определение и назначение ETL-процесса. Примеры инструментальных средств поддержки ETL-процессов. Отличие ETL от ELT.
26. Особенности анализа текстовых данных. Программные средства поддержки анализа текстовых данных.
27. Особенности анализа социальных сетей. Программные средства поддержки анализа социальных сетей.
28. Особенности анализа web-информации и логов. Программные средства поддержки анализа web-информации.
29. Кластеризация и классификация.
30. Поиск ассоциативных правил. Алгоритм «Apriori».
31. Регрессионный анализ.
32. Песочница аналитики данных. Определение, назначение и основные особенности.

33. Оперативная аналитическая обработка данных (OLAP). Многомерный куб.
34. Статистические методы анализа данных. Гипотезы. Ошибка I-рода. Ошибка II-рода.
35. Методы машинного обучения в задачах анализа данных.
36. Линейная классификация.
37. Кластерный анализ.
38. Переосмысление глобальных данных: рост объемов и ценности информации.
39. Система для эффективного сбора, объединения и перемещения больших объемов данных, полученных из различных источников, в централизованное хранилище данных – «Apache Flume».
40. Традиционная архитектура программная системы задачей, которой является анализ данных.

Составил
Доцент, к.т.н., И. В. Никифоров,

19.04.2021

Директор ВШ ПИ *П. Д. Дробинцев*

Никифоров Игорь Валерьевич
Воинов Никита Владимирович
Дробинцев Павел Дмитриевич

Наука о данных и аналитика больших объемов информации

Курсовое проектирование

Учебное пособие

Налоговая льгота — Общероссийский классификатор продукции
ОК 005-93, т. 2; 95 3005 — учебная литература

Подписано в печать	Формат	Печать	
Усл. печ. л.	Уч.-изд. л.	Тираж	Заказ

Отпечатано с готового оригинал-макета, предоставленного авторами,
в Издательско-полиграфическом центре Политехнического университета.
195251, Санкт-Петербург, Политехническая ул., 29.
Тел.: (812) 552-77-17; 550-40-14.